

Appunti Algoritmi di Ottimizzazione

"Introduzione alla Ricerca Operativa"

Introduzione alla Ricerca Operativa

X

Introduzione alla Ricerca Operativa: terminologia, notizie storiche e ambiti di applicazione.

X

1. Ottimizzazione e Ricerca Operativa

Inconsciamente eseguiamo delle *"ottimizzazioni"* diverse volte al giorno, in un qualsiasi contesto funzionale. Molto spesso sono soggetti a dei vincoli specifici, che ci *"limitano"* la nostra area di ricerca delle scelte.

Questa tipologia di situazione viene coniata col termine di *"ricerca operativa"* - usata per la prima volta nel 1939 - ossia una scienza che si occupa di trovare modi di risolvere *problemi decisionali*. Storicamente questo termine viene dal contesto militare, ovvero nella 2GM i responsabili militari inglesi avevano incaricati degli scienziati per trovare un modo di piazzare dei radar nella maniera più efficiente.

Oggi la *ricerca operativa* è la branca della *matematica applicata* in cui *problemi decisionali* complessi vengono analizzati e risolti mediante *modelli matematici e metodi quantitativi avanzati*, con supporto alle decisioni stesse.

Questa ha tantissime applicazioni, come nella *logistica, progettazione di reti, finanza, organizzazione aziendale* e anche ultimamente nel *data science*.

X

2. Esempi di Problemi Decisionali

Di solito un *problema decisionale* ha la seguente struttura:

- Vogliamo *minimizzare* o *massimizzare* una *funzione obiettivo*
- Abbiamo dei *vincoli* su delle variabili
- Ci sono le *variabili decisionali* che viene applicata sulla funzione obiettivo

ESEMPIO. (*KP-Knapsack Problem*)

Si vuole realizzare una *compilation ideale* avendo a disposizione dei file musicali e un CD-ROM dalla capacità di *800 MB*.

L'indice di *gradimento* (in una scala da 1 a 10) e l'*ingombro* in MB di ogni file sono riportati nella tabella seguente:

Canzone	Gradimento	Ingombro (MB)
Light my fire	8	210
Fame	7	190
I will survive	9	230
Imagine	8	195
Let it be	7,5	220
I feel good	7	250

Nel nostro caso abbiamo *sei variabili decisionali*, ovvero x_i corrisponde a "*se mettere la canzone i-esima o meno*". Pertanto, denotando ω col vettore di gradimento ho la funzione obbiettivo $f(x) = \omega^T x$. Tuttavia, denotando con p col vettore dei pesi ho il vincolo $p^T x \leq 800$.

Notiamo che lo step cruciale è quello di capire *quali variabili decisionali* prendere in considerazione.

Concetti Introduttivi della Ricerca Operativa

X

Concetti Introduttivi della Ricerca Operativa. Definizione di un problema di ottimizzazione, insieme delle soluzioni, insieme delle soluzioni ammissibili, funzione obbiettivo. Definizione di soluzione ottimale. Lemma: equivalenza tra i problemi di ottimizzazione. Classificazione dei problemi di ottimizzazione. Nozioni sugli insiemi convessi: definizione di insieme convesso, punto estremo. Lemma: l'intersezione dei insiemi convessi è convessa.

X

0. Voci correlate

- [Introduzione alla Ricerca Operativa](#)

1. Problemi di Ottimizzazione

OBBIETTIVO. Dare un formalismo matematico ai c.d. "*problemi di ottimizzazione*".

#Definizione

Definizione (problema di ottimizzazione).

Sia E un insieme (detto di *soluzioni*), sia $F \subseteq E$ (detto l'*insieme delle soluzioni ammissibili*). La relazione $x \in F$ si dice *vincolo*. Sia funzione $f : E \rightarrow \mathbb{R}$ (detta di *obbiettivo*) la quantità da *minimizzare* o *massimizzare*.

Allora (f, E, F) si dice *problema di ottimizzazione*; ossia devo trovare il minimo/massimo di f su E , vincolato in F . In altre parole, un *problema di ottimizzazione* è una forma del tipo

$$\left(\min_{\mathbf{x} \in E} / \max_{\mathbf{x} \in E} \right) f(\mathbf{x}) \text{ s.t. } \mathbf{x} \in F$$

#Definizione

Definizione (ottimo e valore ottimale di un problema di ottimizzazione).

Sia (f, E, F) un *problema di ottimizzazione*.

Un elemento $\mathbf{x}^*$ tale che $\forall \mathbf{x} \in F, f(\mathbf{x}^*) \leq f(\mathbf{x})$ per un problema di minimizzazione (o $\geq$ per un problema di massimizzazione) si dice l'*ottimo* o la *soluzione ottimale*.

Il valore corrispondente $v := f(\mathbf{x}^*)$ si dice *valore ottimale*.

#Osservazione

Osservazione (cardinalità degli ottimi e valori ottimali).

In un problema di ottimizzazione "*risolvibile*" (vedremo di definire bene questa nozione dopo), ovvero che presenta un *ottimo finito*, si ha che $\exists!$ v valore ottimale ma non è detto che $\mathbf{x}^*$ sia unica.

#Lemma

Lemma (equivalenza dei problemi).

Un problema di *minimizzazione* (f, E, F) è trasformabile in un problema di *massimizzazione* $(\bar{f}, E, F)$.

#Dimostrazione

DIMOSTRAZIONE del Lemma 4

Banalmente si pone $\bar{f} = -f$. ■

Il Lemma 4 ci dice che possiamo, senza perdere di generalità, definire un *problema di ottimizzazione* (f, E, F) senza specificare se è di *massimizzazione* o di *minimizzazione*.

Adesso vediamo come *classificare* i problemi di ottimizzazione.

#Definizione

Definizione (classificazione dei problemi di ottimizzazione).

Sia (f, E, F) un *problema di ottimizzazione*.

Se $E = \mathbb{R}^n$, allora abbiamo (f, E, F) è un *problema di ottimizzazione continua*. Se $E = \mathbb{Z}^n$ allora è *discreta*. Se è un caso misto, ossia ho un prodotto del tipo $E = \mathbb{R}^d \times \mathbb{Z}^{d'}$, allora è un problema di ottimizzazione *mista*.

In particolare, se $F \subset E$ (notare l'inclusione *stretta*!) allora (f, E, F) è un *problema di ottimizzazione vincolata*. Altrimenti se $F = E$ allora è *non vincolata*.

Inoltre, supponendo (f, E, F) discreta se ho che $F \subset \{0, 1\}^n$ allora (f, E, F) è un *problema di ottimizzazione binaria*.

#Osservazione

Osservazione (i problemi di ottimizzazione hanno una "gerarchia").

Notiamo che i problemi di ottimizzazione continue implicano problemi di ottimizzazioni discrete. Pertanto vedremo *prima* il caso continuo, e successivamente il caso discreto.

X

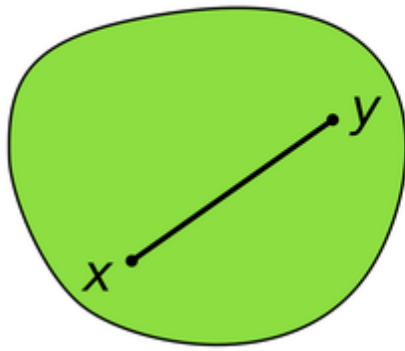
2. Generalità sugli Insiemi Convessi

#Definizione

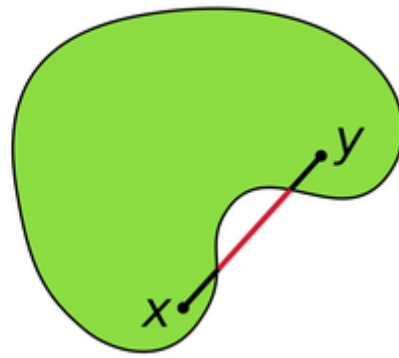
Definizione (insieme convesso).

Un insieme si dice *convesso* se per ogni coppia di punti dell'insieme, il segmento che li congiunge è interamente contenuto nell'insieme.

Convex set



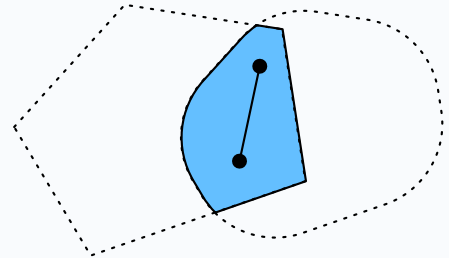
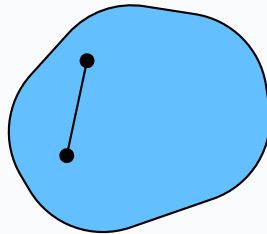
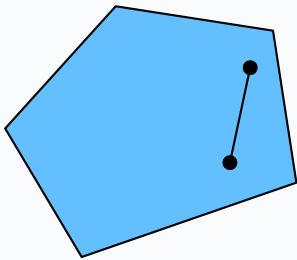
Non-convex set



#Lemma

Lemma (l'intersezione di insiemi convesso è convessa).

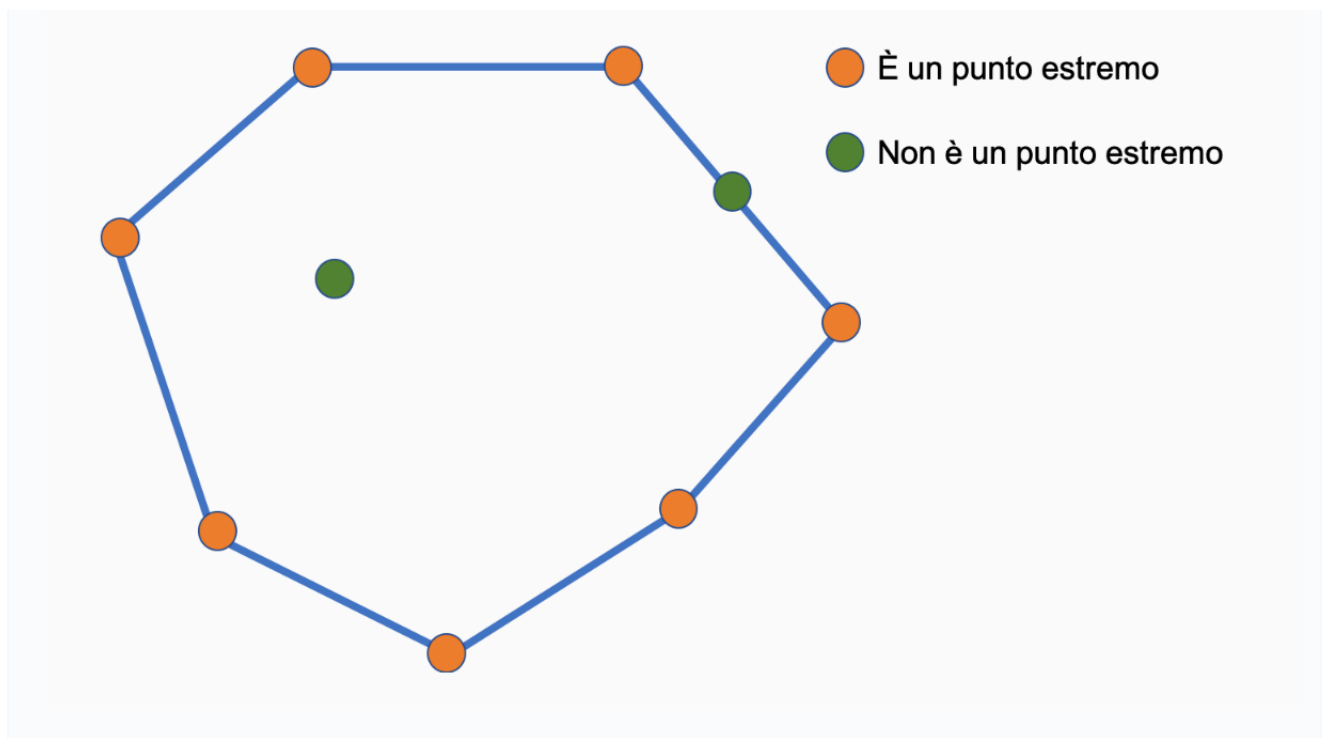
L'intersezione di insiemi convessi è un insieme convesso.



#Definizione

Definizione (punto estremo).

Sia E un insieme convesso. Un *punto estremo* di E è un punto dell'insieme che non appartiene ad alcun segmento congiungente di altri due punti dell'insieme.



"Programmazione Lineare"

Formulazione di Problemi Programmazione Lineare

X

Formulazione dei problemi PL. Prime proprietà di un problema PL.

X

0. Voci correlate

- Algoritmi di Ottimizzazione

1. Definizione di Problema di Programmazione Lineare

#Definizione

Definizione (problema di programmazione lineare).

Un *problema di ottimizzazione* (f, E, F) si dice *di programmazione lineare* (o più semplicemente PL) se:

- La funzione obiettivo f è *lineare e non-degenere*, ossia sse $\exists \omega \in \mathbb{R}^n$ tale che $f(\mathbf{x}) = \omega^T \mathbf{x}$ (teorema di Riesz, 1).
- L'insieme delle soluzioni ammissibili F è definita da *vincoli lineari*, ossia di forma $\mathbf{a}^T \mathbf{x} \leq (=, \geq) \gamma$ dove γ è uno scalare.

#Osservazione

Osservazione (formulazione standard di un problema PL).

Notiamo (e poi dimostreremo) che ogni problema di programmazione lineare può essere formulato come la seguente:

$$\begin{aligned} z &= \max / \min \mathbf{c}^T \mathbf{x} \\ \mathbb{A} \mathbf{x} &\leq \mathbf{b} \\ \mathbf{x} &\geq 0 \end{aligned}$$

Dove $\mathbf{c} \in \mathbb{R}^d$, $\mathbf{b} \in \mathbb{R}^c$ e $\mathbb{A}$ è una matrice $M_{c,d}(\mathbb{R})$ (dove c è il numero dei vincoli imposti).

D'ora in poi denoteremo un problema di *programmazione lineare* con la terna $(\mathbf{c}, \mathbb{A}, \mathbf{b})$.

X

2. Prime Proprietà di Problemi PL

Elenciamo delle prime proprietà di un problema PL $(\mathbf{c}, \mathbb{A}, \mathbf{b})$.

#Proposizione

Proposizione (proporzionalità di un PL).

Sia $(\mathbf{c}, \mathbb{A}, \mathbf{b})$ un problema PL. Allora la funzione obiettivo $z = \mathbf{c}^T \mathbf{x}$ dev'essere lineare rispetto ad ogni variabile decisionale $x_i = \pi_i(\mathbf{x})$. In altre parole, fissata $i \in [1, d] \cap \mathbb{N}$ e se $x_i \neq 0 \wedge \forall j \neq i, x_j = 0$ allora deve valere che $A_{ji}x_i \leq b_i \forall j$.

Riformulando in termini di gradienti, ho che:

$$\begin{aligned} \frac{\partial z}{\partial x_i} &= c_i \\ \frac{\partial b_j}{\partial x_i} &\geq a_{ji} \quad \forall j, i \end{aligned}$$

Una interpretazione della proposizione sopra è che la misura marginale della "efficienza" e l'uso marginale di ogni risorsa devono essere *costanti* sull'intervallo di variazione dei livelli di ogni attività.

Vediamo un'esempio di programmazione in cui non vale questa proprietà, da cui non è lineare

#Esempio

Esempio (carica fissa).

Se abbiamo un problema con una "*carica fissa*", ossia definisco la funzione obbiettivo

$$f(x) = \begin{cases} 0, & x = 0 \\ K + cx, & x > 0 \end{cases}$$

Chiaramente $\partial_x(f)(x) \neq c \forall x \in [0, +\infty)$ (ossia non è costante nella regione ammissibile), da cui evinciamo che questo non è un problema PL.

#Proposizione

Proposizione (additività).

Sia $(\mathbf{c}, \mathbb{A}, \mathbf{b})$ un problema PL. Allora la funzione obbiettivo z dev'essere una combinazione lineare tra $\mathbf{x}, \mathbf{c}$. In altre parole, le "*attività*" $\mathbf{x}$ sono "*separabili*".

#Proposizione

Proposizione (divisibilità).

Si ha che $\forall j, x_j \in \mathbb{R}$. Cioè escludiamo i problemi di programmazione lineari *complessi*.

#Proposizione

Proposizione (certezza).

Si ha che i *coefficienti* del problema $(\mathbf{c}, \mathbb{A}, \mathbf{b})$ sono valori *conosciuti*, ossia non si basano su *distribuzioni di probabilità* o robe del genere.

Possibili Esiti di Un Problema PL

X

Possibili Esiti di un problema PL: Innammissibilità del problema, esistenza di un'unica soluzione ottima, esistenza di infinite soluzioni ottime, inesistenza di soluzioni ottime. Esempi di ogni caso.

X

0. Voci correlate

- Formulazione di Problemi Programmazione Lineare

1. Possibili Esiti di Un Problema PL

#Lemma

Lemma (esiti possibili di un problema PL).

Sia $(\mathbf{c}, \mathbb{A}, \mathbf{b})$ un problema PL, ossia

$$z = \max / \min \mathbf{c}^T \mathbf{x} \\ X : \{\mathbb{A}\mathbf{x} \leq \mathbf{b}, \mathbf{x} \geq 0\}$$

Allora sicuramente si ha uno dei casi:

- $X = \emptyset$, ossia la soluzione ammissibile è vuota. In altre parole, non possiamo cercare nessun tipo di soluzione
- Esiste una soluzione ottima, detta $\mathbf{x}^*$.
- Esistono infinite soluzioni ottime; questo succede se la funzione obiettivo è illimitata in X
- Non esistono soluzioni ottime; dunque, a seconda della natura del problema si pone $z = \pm\infty$.

#Osservazione

Osservazione (soluzione e valore unico hanno cardinalità diverse).

Notiamo che le soluzioni potrebbero essere infinite (li chiamiamo $(\mathbf{x}_n)_n$), tuttavia il *valore ottimo* z rimane comunque unico.

#Osservazione

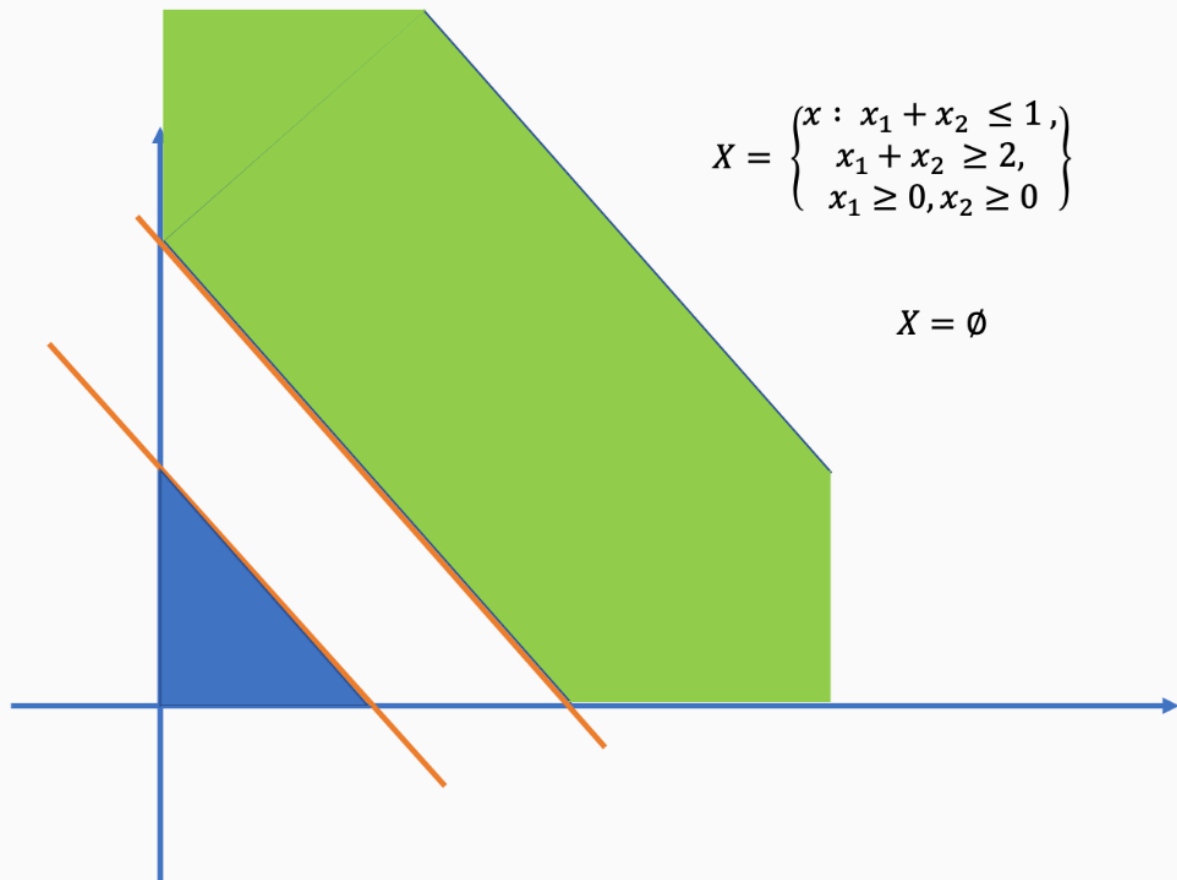
Osservazione (le soluzioni o sono uniche o sono infinite).

Notiamo che c'è una specie di aut-aut tra l'unicità e l'infinità delle soluzioni. Questo è dovuto alla *convessità della regione* X (per ora la diamo buona, ne parleremo bene successivamente), infatti supponendo per assurdo avrei che se solamente $\mathbf{x}_1, \mathbf{x}_2$ sono entrambe soluzioni nella regione ammissibile X allora esiste una retta congiungente $\gamma : [0, 1] \rightarrow X$, tale che $\gamma(0) = \mathbf{x}_1 = \gamma(1) = \mathbf{x}_2 = \gamma(t), \forall t$. Tuttavia $\gamma(x \in (0, 1))$ ci fornisce un range infinito di valori per la soluzione ottimale, che è l'assurdo e concludendo.

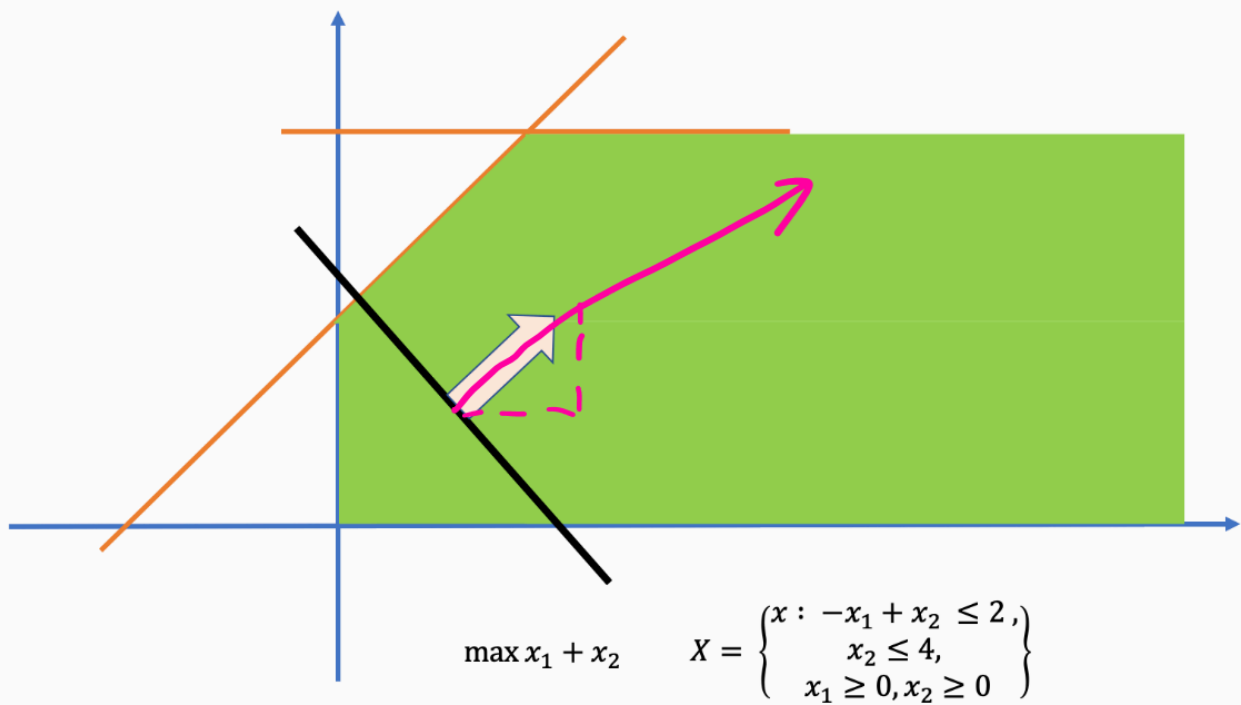
2. Esempi di Ogni Caso

Diamo una figura illustrativa in cui abbiamo un'esito di un problema PL.

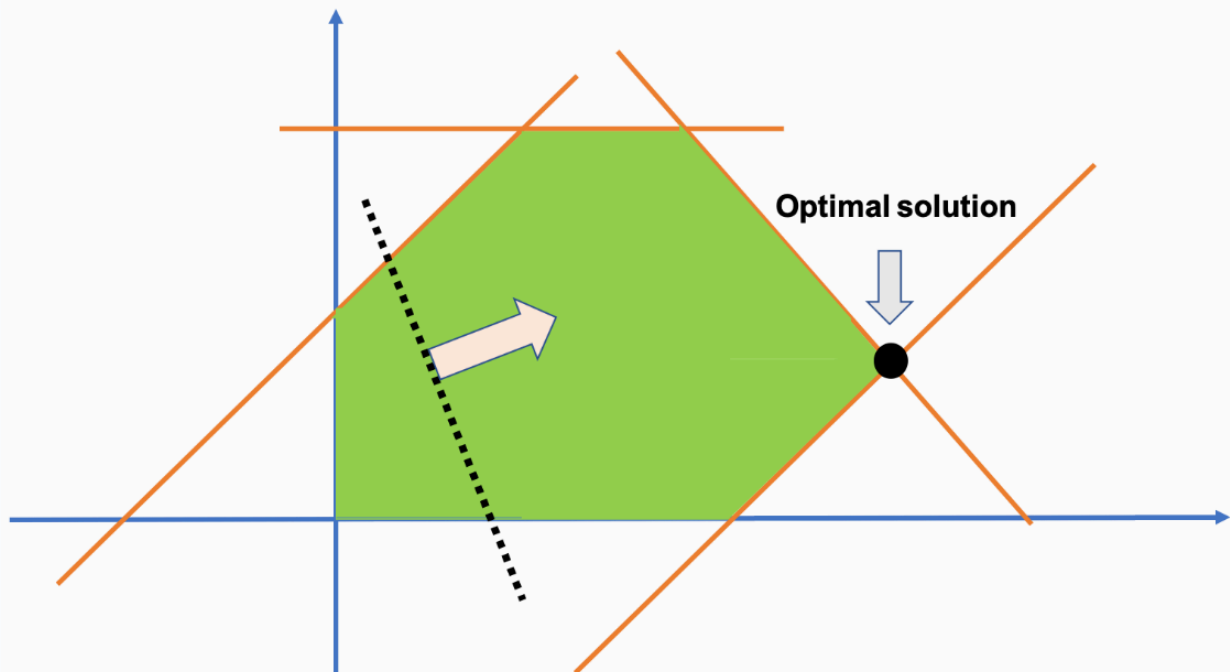
La regione ammissibile è vuota



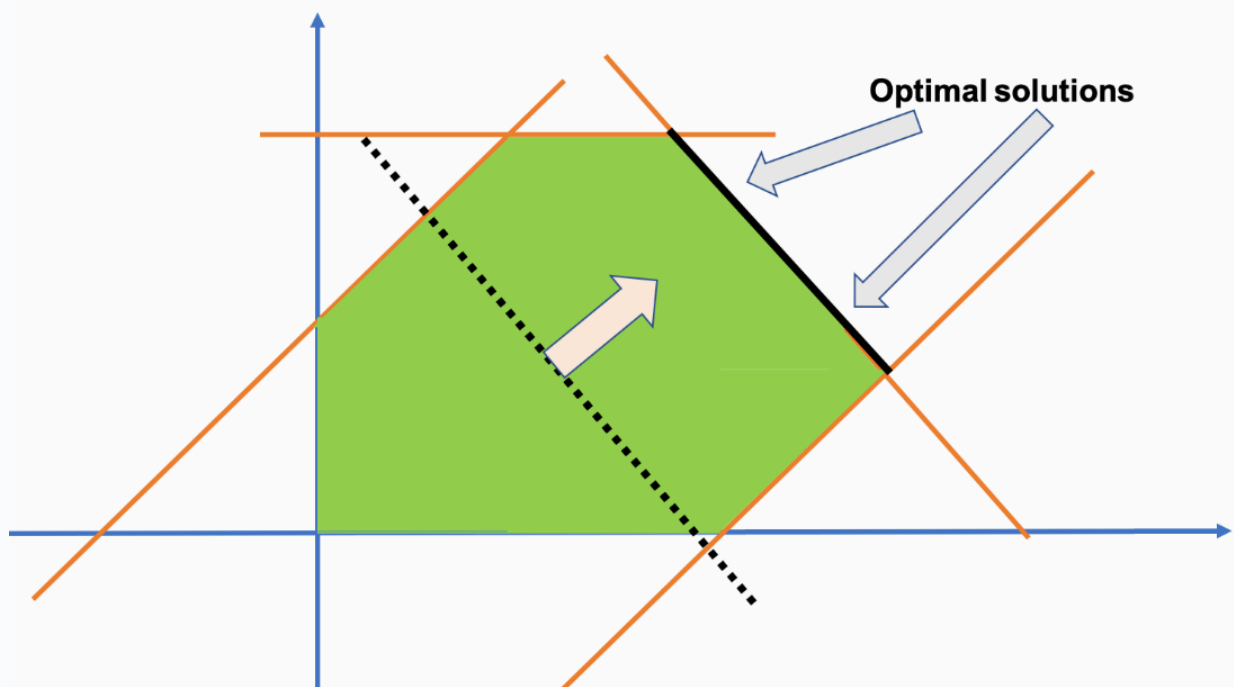
Non Esistenza di un Valore Ottimo



Esistenza di un'Unica Soluzione Ottima



Esistenza di Soluzioni Ottime Infinite



Teorema Fondamentale dei Problemi PL

X

Teorema fondamentale dei problemi PL: la soluzione ottima è su un vertice. Lemmi preliminari, enunciato, dimostrazione, applicazione pratica.

▼

0. Voci correlate

- Formulazione di Problemi Programmazione Lineare
- Possibili Esiti di Un Problema PL

1. Definizioni e Risultati Preliminari

#Definizione

Definizione (iperpiano e semispazi di supporto dell'iperpiano).

Siano $\omega \in \mathbb{R}^d, b \in \mathbb{R}$. L'insieme $H := \{x \in \mathbb{R}^d : \omega^T x = b\}$ si dice *iperpiano*, che viene univocamente identificata come (ω, b) .

Le regioni $X^- := \{x \in \mathbb{R}^d : \omega^T x \leq b\}$ e $X^+ := \{x \in \mathbb{R}^d : \omega^T x \geq b\}$ si dicono *semispazi* di supporto H .

#Definizione

Definizione (poliedro convesso).

Diciamo che l'intersezione finita di un numero di *semispazi* e *iperpiani* è un *poliedro convesso*.

#Lemma

Lemma (convessità dei poliedri).

I poliedri convessi sono convessi.

#Definizione

Definizione (poliedro finito).

Un *poliedro* P si dice finito sse $\exists M > 0 : \forall \mathbf{x} \in P, \|\mathbf{x}\| \leq M$. Ovvero, se esiste una palla $B \supseteq P$.

#Lemma

Lemma (le regioni ammissibili di problemi PL sono dei poliedri convessi).

Sia $(\mathbf{c}, \mathbf{A}, \mathbf{b})$ un problema PL. Allora la regione ammissibile X è un poliedro convesso.

Definizione (vertici di un poliedro).

I punti estremi di un poliedro convesso si dicono *vertici*.

X

2. Teorema Fondamentale dei PL

#Teorema

Teorema (teorema fondamentale dei problemi PL).

Sia $(\mathbf{c}, \mathbb{A}, \mathbf{b})$ un problema PL. Supponendo che $X \neq \emptyset$ e che *esista* una soluzione unica, allora vale la seguente condizione necessaria:

$$\forall \mathbf{x}^* \text{ s.t. } \max / \min z = \mathbf{c}^T \mathbf{x}^* \implies \mathbf{x}^* \text{ è un vertice di } X$$

In altre parole, le soluzioni al problema stanno sicuramente sui vertici.

#Dimostrazione

DIMOSTRAZIONE del Teorema 7

Si tratta di usare il fatto che X è un poliedro finito e dunque un insieme convesso.

Facciamo una dimostrazione per assurdo. Assumiamo che ci sia l'unica soluzione ottima $\mathbf{x}^*$ non stia su una vertice. Ciò implica che devono esistere due soluzioni ammissibili diverse $\mathbf{x}_1, \mathbf{x}_2$ tali che la loro retta congiungente contenga $\mathbf{x}^*$. Formuliamo questo come la seguente combinazione convessa:

$$\exists \alpha \in (0, 1) : \mathbf{x}^* = \alpha \mathbf{x}_1 + (1 - \alpha) \mathbf{x}_2$$

Pertanto i loro valori corrispondenti sono

$$z^* = \alpha z_2 + (1 - \alpha) z_1$$

Siccome $\alpha + 1 - \alpha = 1$, abbiamo tre possibilità:

- $z^* = z_2 = z_1$: in questo caso si ha l'assurdo per cui la soluzione ottima è unica
- $z_1 < z^* < z_2$: in tal caso z^* non è più ottima
- $z_2 < z^* < z_1$: caso analogo a prima

Concludendo la dimostrazione. ■

Questo teorema è utile da un punto di vista pratico, siccome per implementare un algoritmo di *ricerca* del valore ottimale x^* è sufficiente "*navigare*" sui vertici di X . Infatti l'idea dell'algoritmo del simplesso si basa su "*navigare*" la frontiera della regione ammissibile X , passando da un vertice all'altro per raggiungere la soluzione ottimale.

Forma Standard dei Problemi PL

X

Breve descrizione qui

X

0. Voci correlate

- Formulazione di Problemi Programmazione Lineare

1. Forma Standard di Problemi PL

#Definizione

Definizione (forma standard di un problema PL).

Un problema PL $(\mathbf{c}, \mathbb{A}, \mathbf{b})$ si dice *di forma standard* se ha la seguente forma:

$$\begin{cases} \max / \min z = \mathbf{c}^T \mathbf{x} \\ \mathbb{A} \mathbf{x} = \mathbf{b} \\ \mathbf{x} \geq \underline{0} \end{cases}$$

Inoltre vale che $\mathbf{d} > \mathbf{c}$.

Vediamo che questa si dice *forma standard*, in quanto ogni *problema PL* può essere ricondotto ad una sua forma standard. Per enunciare questo teorema facciamo la seguente osservazione:

#Osservazione

Osservazione (trasformare una disequazione in un'equazione).

Come trasformiamo una disequazione del tipo

$$x \leq 5$$

In un'equazione "="? Si può associare alla disequazione una quantità non-negativa detta di *"slack"*, che misura la distanza di x da 5. Naturalmente $x \leq 5 \iff 5 - x \geq 0$.

Ossia, per ho $x + s = 5 \iff x \leq 5$ per $s \geq 0$.

In un certo senso, imporre $s = 0$ è equivalente a imporre che $x = 5$, ossia il punto si muove sul vincolo.

#Osservazione

Osservazione (invertire il segno di disequazione).

Per convertire una disequazione del tipo

$$x \geq 10$$

Faccio una procedura simile, solo che ci *sottraggo* la variabile aggiuntiva. In questo caso si chiama *variabile di surplus*. Allora ho

$$\forall s \geq 0, x - s = 10$$

#Teorema

Teorema (canonicità della forma standard dei problemi PL).

Per ogni PL $(\mathbf{c}, \mathbb{A}, \mathbf{b})$ esiste una PL equivalente in forma canonica $(\mathbf{c}, \mathbb{A}, \mathbf{b})'$.

Ricordiamo che la *un PL è in forma standard* sse i coefficienti $\mathbb{A}$ ha una permutazione tale che $P\mathbb{A} = (A \mid \mathbb{1})$.

Soluzioni di Base e Fuori Base

X

Osservazioni sui problemi PL in termini dell'algebra lineare. Definizione della matrice delle colonne in base e fuori base, soluzioni in base e fuori base.

X

0. Voci correlate

- [Formulazione di Problemi Programmazione Lineare](#)

1. Soluzioni di Base e Fuori Base

Osserviamo che con un problema PL di forma standard

$$\begin{cases} \max / \min z = \mathbf{c}^T \mathbf{x} \\ \mathbb{A} \mathbf{x} = \mathbf{b} \\ \mathbf{x} \geq \underline{0} \end{cases}$$

Possiamo supporre, senza perdere di generalità, che $\text{rg } \mathbb{A} = c$ (ricordando che $d > c$ e che valgono [le proprietà del rango](#)). Infatti, se ci fossero più righe linearmente dipendenti tra di loro, possiamo *"buttarli via"* in quanto descrivono lo stesso vincolo. Allora riordinando le colonne possiamo *"decomporre"* la matrice $\mathbb{A}$ come

$$\mathbb{A} := \mathbf{B} \mid \mathbf{N}$$

dove $\mathbf{B}$ è una matrice $c \times c$ non singolare detta *matrice delle colonne in base*, e $\mathbf{N}$ è una matrice $c \times (d - c)$ detta *matrice delle colonne fuori base*.

Analogamente possiamo partizionare una soluzione $\mathbf{x}$ in due parti: avrà c componenti, detti *in base*, invece le restanti $(d - c)$ sono detti componenti *fuori base*. Ossia $\mathbf{x} = (\mathbf{x}_B | \mathbf{x}_N)^T$.

Così abbiamo che

$$\mathbb{A}\mathbf{x} = \mathbf{b} \iff (\mathbf{B} | \mathbf{N}) \begin{pmatrix} \mathbf{x}_B \\ \mathbf{x}_N \end{pmatrix} = \mathbf{b} \iff \mathbf{B}\mathbf{x}_B + \mathbf{N}\mathbf{x}_N = \mathbf{b} \iff \mathbf{x}_B = \mathbf{B}^{-1}(\mathbf{b} - \mathbf{N}\mathbf{x}_N)$$

Notiamo che $\mathbf{x}_N$ è un vettore *"libero"*, nel senso che $\mathbb{A}\mathbf{x} = \mathbf{b}$ è risolto fissando arbitrariamente $d - \text{rg } \mathbb{A}$, ovvero proprio il vettore $\mathbf{x}_N$. Pertanto un modo *"intelligente"* per porre $\mathbf{x}_N$ è quello di renderlo nullo, così che ammaziamo il prodotto $\mathbf{N}\mathbf{x}_N \equiv 0$. Ossia si ha

$$\mathbf{x}_B = \mathbf{B}^{-1}\mathbf{b}$$

Notiamo che in tal caso questa soluzione è ammissibile sse $\mathbf{B}^{-1}\mathbf{b} \geq 0$. In tal caso diciamo che la soluzione $\mathbf{x} = (\mathbf{x}_B | \mathbf{0})^T$ è una *soluzione di base ammissibile* (BFS).

Notiamo che, data la base $\mathbf{B}$ scelta possiamo esplicitamente calcolarne il suo valore target:

$$z_B = \mathbf{c}^T \mathbf{x} = (\mathbf{c}_B^T \quad \mathbf{c}_N^T) \begin{pmatrix} \mathbf{x}_B \\ \mathbf{x}_N \end{pmatrix} = \mathbf{c}_B^T \mathbf{x}_B = \mathbf{c}_B^T \mathbf{B}^{-1} \mathbf{b}$$

Ossia ho una forma quadratica.

Soluzioni Vertici Ammissibili

X

*Definizione di frontiera di un vincolo, soluzioni vertice ammissibile e non ammissibile.
Definizione di soluzioni CPF adiacenti, spigoli. Proprietà delle soluzioni CPF. Test dell'ottimalità.*

X

0. Voci correlate

- Formulazione di Problemi Programmazione Lineare
- Teorema Fondamentale dei Problemi PL

1. Definizione di Soluzioni CPF

Sia $(\mathbf{c}, \mathbb{A}, \mathbf{b})$ un problema PL d -dimensionale a c vincoli. Sappiamo che ogni *soluzione* deve stare in un *vertice* della regione ammissibile X . Adesso vediamo di *formulare* il concetto delle soluzioni CPF, ossia le soluzioni vertici ammissibili.

#Definizione

Definizione (frontiera vincolare).

Una *frontiera vincolare* è una linea che forma la *frontiera* di uno dei vincoli. Ossia è una riga di $\mathbb{A}\mathbf{x} = \mathbf{b}$.

#Definizione

Definizione (soluzione vertice).

Sia $(\mathbf{c}, \mathbb{A}, \mathbf{b})$ un problema PL d dimensioni e c vincoli. Una soluzione $\mathbf{x}$ si dice *vertice* se risolve un'intersezione di d vincoli.

In particolare se una soluzione ammissibile allora si dice che è CPF.

Un modo di determinare *tutte le soluzioni vertice* è quello semplicemente di risolvere tutti i possibili sistemi lineari con rango d , fornite da combinazioni di vincoli. Vediamo infatti il seguente teorema di caratterizzazione.

#Teorema

Teorema (teorema di caratterizzazione dei vertici).

Una soluzione $\mathbf{x}$ in un problema PL $(\mathbf{c}, \mathbb{A}, \mathbf{b})$ standard è *CPF* del poliedro $P(\mathbb{A}, \mathbf{b})$ sse è una *BST*. Ovvero

$$\mathbf{x} \text{ è CPF} \iff \mathbf{x} = (\mathbf{x}_B \mid \underline{0}) \text{ s.t. } \mathbf{B}^{-1}\mathbf{b} \geq 0$$

Ovvero scegliendo *una base di colonne* $\mathbb{A}$, questa moltiplicata per $\mathbf{b}$ dev'essere maggiore di zero.

#Dimostrazione

DIMOSTRAZIONE del Teorema 3

" $\implies$ ": Chiaramente un punto di un poliedro è un *vertice* sse soddisfa l'uguaglianza di d vincoli linearmente indipendenti. Allora certamente soddisfa $\mathbf{B}\mathbf{x}_B = \mathbf{b}$ (ovviamente fissando $\mathbf{x}_N = \underline{0}$).

" $\impliedby$ ": Per definizione ogni BFS soddisfa $\mathbf{x} = \mathbf{B}^{-1}\mathbf{b} \geq 0$, che sono $d - c$ vincoli. Inoltre ogni BFS soddisfa $\mathbb{A}\mathbf{x} = \mathbf{b}$, che produce c vincoli. Tutti i vincoli sono linearmente indipendenti, siccome la matrice dei coefficienti $\mathbb{A}$ è della forma

$$\begin{pmatrix} \mathbf{B} & \mathbf{N} \\ \underline{0} & \mathbf{I}_{d-c} \end{pmatrix}$$

Ciò vuol dire che ogni BFS soddisfa d vincoli, da cui $\mathbf{x}$ è una CPF. ■

2. Soluzioni CPF Adiacenti

Intuizione. Notiamo che due *soluzioni CPF* $\mathbf{x}_1, \mathbf{x}_2 \in \mathbb{R}^2$ potrebbero *condividere* un vincolo. Chiameremo questi punti come *adiacenti*. Generalizziamo questa nozione su $d \in \mathbb{N}$.

#Definizione

Definizione (soluzioni CPF adiacenti).

Due soluzioni $\mathbf{x}_1, \mathbf{x}_2$ CPF - di un problema PL a d dimensioni e c vincoli - si dicono *adiacenti* sse condividono $d - 1$ vincoli. Ossia sono connessi da un *iperpiano* che viene determinato dalle intersezioni dei vincoli condivisi; tale iperpiano si dice *spigolo* della regione ammissibile.

Un modo per determinare *quali* soluzioni CPF sono adiacenti è quella di fissare una soluzione vertice e

Vediamo un paio di proprietà delle soluzioni CPF adiacenti.

#Proposizione

Proposizione (proprietà delle soluzioni adiacenti).

Sia $(\mathbf{c}, \mathbb{A}, \mathbf{b})$ un problema PL. Allora:

- Se esiste esattamente un'unica soluzione $\mathbf{x}^*$, allora dev'essere CPF
- Se esistono più soluzioni (e una regione ammissibile limitata), allora almeno due devono essere soluzioni CPF adiacenti.

#Dimostrazione

DIMOSTRAZIONE della [Proposizione 5](#)

Omettiamo il primo punto, in quanto è stato già dimostrato (vedere [qui](#)). Diamo l'idea della dimostrazione per il secondo punto.

Notiamo che da un punto di vista geometrico, risolvere z vuol dire far

"aumentare/diminuire" l'iperpiano rappresentato da z (ossia $\mathbf{c}^T \mathbf{x}$), finché va a contenere il segmento che connette le soluzioni CPF. Di conseguenza, tutte le soluzioni ottime possono essere ottenute come medie pesate delle soluzioni CPF. ■

#Proposizione

Proposizione (la finitezza dei punti CPF).

Per ogni $(\mathbf{c}, \mathbb{A}, \mathbf{b})$ il poliedro $P(\mathbb{A}, \mathbf{b})$ contiene un punto finito di vertici. Ossia esistono un numero finito di soluzioni CPF.

#Dimostrazione

DIMOSTRAZIONE del [Proposizione 6](#)

Siccome ogni CPF è determinata da l'intersezione di d vincoli, abbiamo che possiamo scegliere $d + c$ vincoli su d . Ovvero, supponendo che tutti i vincoli non siano paralleli (generando dunque sempre dei vertici diversi), ho il coefficiente binomiale

$$\binom{d+c}{d} = \frac{(d+c)!}{d!c!}$$

Ovviamente questa è finita, anche se cresce in una maniera fattoriale. ■

#Osservazione

Osservazione (motivazione per l'algoritmo del simplesso).

Notiamo che per trovare una soluzione ottimale con *"forza bruta"* (brute force), si dovrebbe calcolare una quantità enorme di punti in funzione di z . Da questo sorge la necessità di trovare algoritmi che siano in grado di esaminare i punti *"intelligentemente"* e trovare la soluzione ottima in *"pochi"* (da un punto di vista asintotico) passi nei confronti della forza bruta.

Algoritmo del Simplexso

X

Algoritmo del simplexso: idea, fase II, fase I.

X

0. Voci correlate

- [Soluzioni di Base e Fuori Base](#)
- [Teorema Fondamentale dei Problemi PL](#)

1. Idea dell'algoritmo del Simplexso

Q. Dal teorema fondamentale dei problemi PL sappiamo che per trovare il punto ottimale basta scegliere il *punto CPF* opportuno. Come facciamo a trovarlo in una maniera efficiente?

A. Usiamo il fatto che l'insieme delle soluzioni ammissibili F è concava, da cui si ha il seguente test:

Proposizione (test dell'ottimalità).

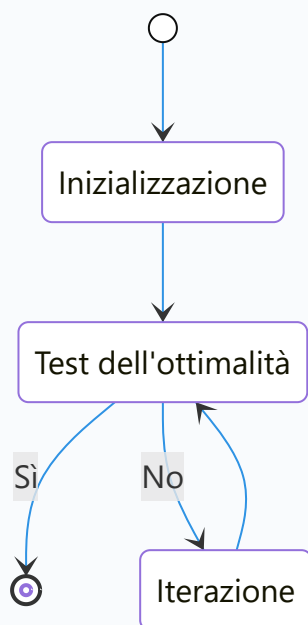
Consideriamo un problema PL che abbia *almeno* una soluzione ottima.

Se una soluzione CPF non ha soluzioni CPF adiacenti migliori, allora quest'ultima dev'essere la soluzione migliore.

L'idea dell'algoritmo del simplesso consiste nella seguente iterazione:

- Inizializzare il CPF iniziale come $(0, 0)$.
- Effettuare un test dell'ottimalità sul CPF corrente
- Se altri punti sono peggiori, allora concludere che il CPF corrente è la migliore (condizione di arresto)
- Se esiste un punto migliore, allora spostarsi verso lì
- Ripetere finché non si ha la condizione di arresto

Ossia l'algoritmo è descritto dal seguente diagramma dei stati:



2. Algoritmo del Simplexso: Fase II

Partiamo da un *esempio* di un problema PL, e lo risolviamo usando un algoritmo noto come l'*algoritmo del simplesso*.

Supponiamo di avere il PL

$$\begin{aligned}
\max z &= 3x_1 + 5x_2 \\
x_1 &\leq 8 \\
2x_2 &\leq 12 \\
3x_1 + 2x_2 &\leq 18 \\
x_1, x_2 &\geq 0
\end{aligned}$$

Che, introducendo le *variabili slack* ha forma standard

$$\begin{aligned}
\max z &= 3x_1 + 5x_2 \\
x_1 + s_1 &= 4 \\
2x_2 + s_2 &= 12 \\
3x_1 + 2x_2 + s_3 &= 18 \\
x_1, x_2, s_1, s_2, s_3 &\geq 0
\end{aligned}$$

Come faccio a trovare la prima BFS, ovvero le *soluzioni in base*? Scelgo le variabili *in base* come s_1, s_2, s_3 e dunque ponendo $(x_1, x_2)^T = \underline{0}$. In questo modo la matrice dei coefficienti $\mathbb{A}$ è composta da $[B|N]$, dove $A = \mathbb{1}$ è la matrice identità. Così, per ricavare s_1, s_2, s_3 risolvo il sistema lineare

$$\mathbb{1} \begin{pmatrix} s_1 \\ s_2 \\ s_3 \end{pmatrix} = \begin{pmatrix} 4 \\ 12 \\ 18 \end{pmatrix}$$

Ovviamente questa è risolta ponendo $(s_1, s_2, s_3) = (4, 12, 18)$ che è una soluzione in base ammissibile. Questo mi giustifica la scelta $(x_1, x_2) = 0$ che è ottenuta ponendo s_1, s_2, s_3 come le variabili *in base*.

Adesso, come faccio a "*spostarmi*" dal punto $(x_1, x_2) = (0, 0)$, trovando un punto adiacente? Posso far semplicemente variare una delle variabili fuori base e tenendo le altre fisse.

Infatti in questo modo vado a risolvere $n - 1$ disequazioni.

Nel nostro caso, ci spostiamo verso la direzione x_2 e dunque ponendo $x_1 = 0$. In tal modo ho il sistema dei coefficienti che diventa

$$\begin{aligned}
x_1 + s_1 &= 4 \implies s_1 = 4 \\
2x_2 + s_2 &= 12 \implies 2x_2 + s_2 = 12 \\
3x_1 + 2x_2 + s_3 &= 18 \implies 2x_2 + s_3 = 18 \\
x_1, x_2, s_1, s_2, s_3 &\geq 0 \implies "
\end{aligned}$$

Riscrivendolo in termini delle *variabili in base* ottengo

$$\begin{aligned}
s_1 &= 4 \implies s_1 = 4 \\
2x_2 + s_2 &= 12 \implies s_2 = 12 - 2x_2 = 2(6 - x_2) \\
2x_2 + s_3 &= 18 \implies s_3 = 18 - 2x_2 = 2(9 - x_2) \\
x_1, x_2, s_1, s_2, s_3 &\geq 0 \implies "
\end{aligned}$$

Far "*muovere x_2 finchè colpisce uno dei vincoli*" corrisponde a trovare x_2 tale che tutti i vincoli s_1, s_2, s_3 siano ancora rispettati. Ossia va a risolvere contemporaneamente tutte le equazioni sopra, col massimo numero possibile. In tal caso ho $x_2 = 6$ (altrimenti se $x_2 = 9$ allora avrei $s_2 < 0$).

Pertanto ho ottenuto la BFS adiacente $(x_1, x_2) = (0, 6)$.

Eseguiamo il test dell'ottimalità:

$$z(0,0) = 0$$

e invece

$$z(0,6) = 0 + 15 = 15$$

Chiaramente $(0,6)$ è una BFS migliore.

Come faccio sapere "*dove posso muovermi*"? Esprimo la funzione obbiettivo in termini delle variabili che non siano state usate per effettuare lo "*spostamento*" (nel nostro caso x_2). In particolare ho che

$$s_2 = 2(6 - x_2) \iff \frac{1}{2}s_2 = 6 - x_2 \iff x_2 = 6 - \frac{1}{2}s_2$$

Così ho

$$z = 3x_1 + 5x_2 = 3x_1 - \frac{5}{2}s_2 + 30$$

Noto che $\frac{\partial(z)}{\partial s_2} < 0$, infatti corrisponderebbe all'idea di "*percorrere lungo il vincolo s_2* ", che nel nostro caso vuol dire semplicemente fare dei passi indietro.

Dato che $\frac{\partial(z)}{\partial x_1} = 3$, posso muovermi lungo l'asse x_1 . Pertanto fisso le variabili *in base* x_2, s_1, s_3 e le *variabili fuori base* $x_1, s_2 = 0$. Riscrivendo il sistema esplicitando le *variabili in base* ottengo

$$\begin{aligned} s_1 = 4 - x_1 &\implies " \\ x_2 = 6 - \frac{1}{2}s_2 &\implies " \\ s_3 = 18 - 3x_1 - 2x_2 &\implies s_3 = 18 - 3x_1 - 12 + s_2 = 6 - 3x_1 + s_2 \\ x_1, x_2, s_1, s_2, s_3 \geq 0 &\implies " \end{aligned}$$

Ponendo $s_2 \equiv 0$ ho

$$\begin{aligned} s_1 = 4 - x_1 &\implies s_1 = (4 - x_1) \\ x_2 = 6 - \frac{1}{2}s_2 &\implies x_2 = 6 \\ s_3 = 6 - 3x_1 + s_2 &\implies s_3 = 6 - 3x_1 = 3(2 - x_1) \\ x_1, x_2, s_1, s_2, s_3 \geq 0 &\implies " \end{aligned}$$

Chiaramente $x_1 = 2$ soddisfa tutte le equazioni rendendo $s_1 = 0 \wedge s_3 \geq 0$ e ottenendo dunque il nuovo punto $(x_1, x_2) = (2, 6)$.

Eseguendo di nuovo il test dell'ottimalità ho

$$z(0,6) = 15 \rightsquigarrow z(2,6) = 6 + 15 = 21$$

Chiaramente questa è migliore. Per vedere se posso spostarmi sulle altre variabili (in questo caso solo il vincolo s_3), esprimo la funzione obbiettivo in termini di variabili "libere" s_2, s_3 .

$$\begin{aligned}
 z &= 3x_1 + 2x_2 \\
 &= 30 + 3x_1 - \frac{5}{2}s_2 \\
 &= 30 + 6 - s_3 + s_2 - \frac{5}{2}s_2 \\
 &= 36 - s_3 - \frac{3}{2}s_2
 \end{aligned}$$

Noto che $\nabla z < \underline{0}$, da cui chiaramente non ho più soluzioni CPF adiacenti migliori, concludendo così l'algoritmo del simplesso. ■

#Osservazione

Osservazione (caso di soluzioni infinite).

Nell'ultimo step, cosa succederebbe se avessi $\exists i : \partial_{s_i}(z) = 0$? Ciò vorrebbe dire che "*muovendosi lungo*" uno dei vincoli la soluzione non cambierebbe, ossia la soluzione CPF adiacente avrebbe lo stesso valore.

Da ciò deduciamo che il problema PL ha soluzioni infinite.

X

3. Algoritmo del Simplexso: Fase I

Q. Osserviamo che non è sempre garantito avere dei vincoli in forma $Ax \leq b$, a volte si potrebbe capitare un vincolo del tipo $ax = b$ o $ax > b$. In questo modo, la soluzione $\underline{0}$ potrebbe non essere più una soluzione *ammissibile*, in quanto si avrebbe che le variabili slack potrebbero essere negative. Come facciamo a gestire questi casi?

#Definizione

Definizione (problema PL in forma canonica).

Un problema PL in forma standard si dice *in forma canonica* se la sua matrice dei coefficienti A si può esprimere come

$$A = [I|H]$$

Dove $I \sim 1$ è una matrice identità.

Non è immediato che ogni problema è in *forma canonica*, da cui $\underline{0}$ non è una soluzione ammissibile per la fase I dell'algoritmo del simplesso. Come facciamo a trovare un punto ammissibile per il problema PL?

L'idea è quella di definire un *problema ausiliare*, che presenta una soluzione ammissibile del problema originale come la sua *soluzione ottimale*.

Un metodo per farlo è di aggiungere alla matrice dei coefficienti le *variabili artificiali* $\mathbf{u}$ con coefficienti 1, e poi di imporre la *funzione obiettivo* come la minimizzazione di $\|\mathbf{u}\|_1$, che è minima sse tutti i valori $u_i = 0$.

$$\begin{aligned}\max z = \mathbf{c}^T \mathbf{x} &\implies \min w = \|\mathbf{u}\|_1 \\ \mathbf{Ax} = \mathbf{b} &\implies \mathbf{Ax} + \mathbf{1u} = \mathbf{b} \\ \mathbf{x} \geq 0 &\implies \mathbf{x}, \mathbf{u} \geq 0\end{aligned}$$

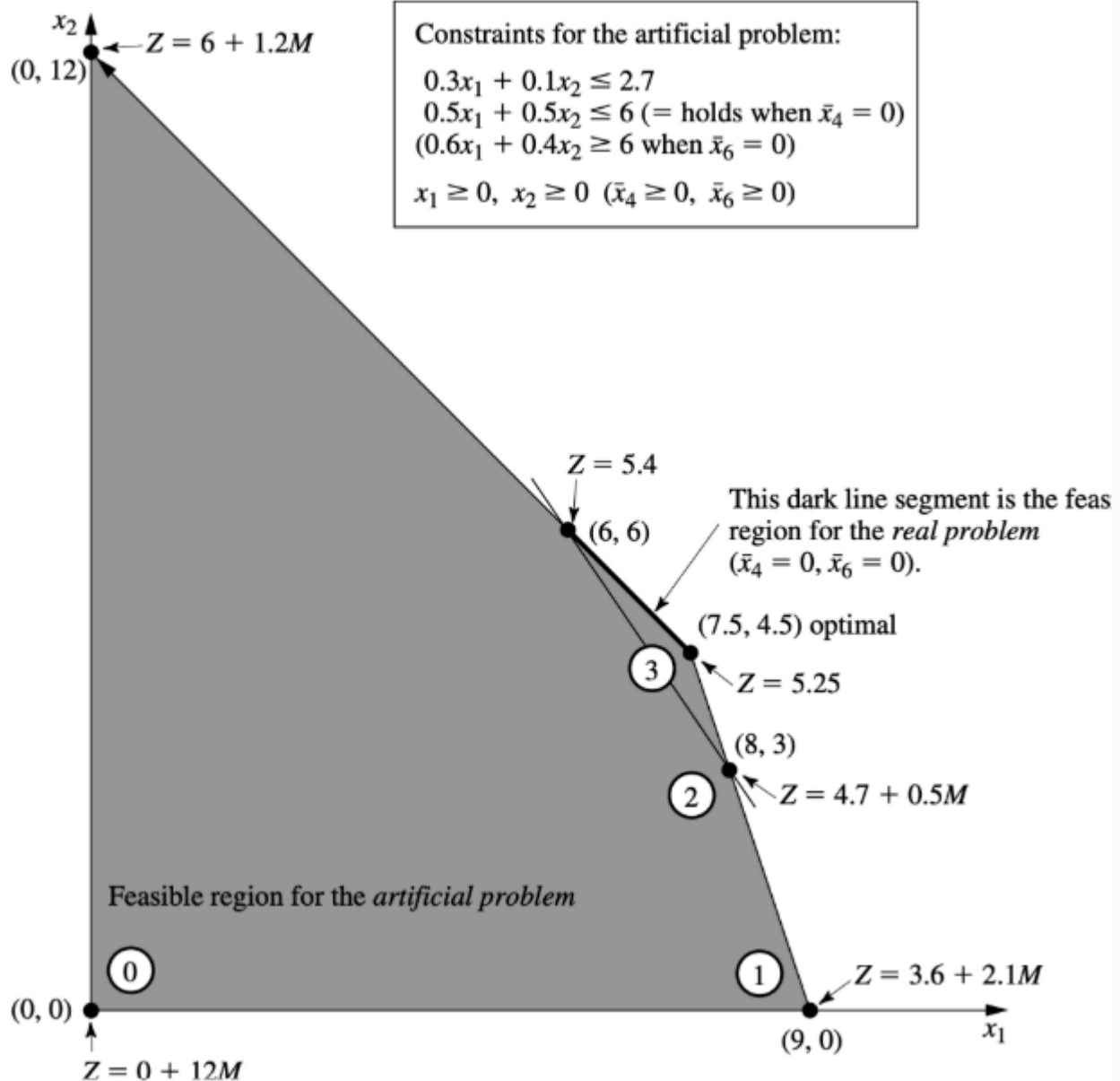
Osserviamo che se $w^* > 0$, ciò vuol dire che una componente $u_i > 0$ (siccome sono tutte non-negative); allora il problema PL originale non presenta soluzioni ammissibili, infatti avrei che $\mathbf{Ax} + \mathbf{1u} \neq \mathbf{b}$.

Una volta trovato un valore ottimale w associato al vettore $\mathbf{x}^*$, posso eseguire l'algoritmo del simplesso iniziando $\mathbf{x}_0 = \mathbf{x}^*$.

Un altro modo per formulare la *funzione obiettivo* è il metodo del "*big M*", ossia ponendo

$$\min z = \mathbf{c}^T \mathbf{x} + M(\|\mathbf{u}\|_1)$$

Dove M è un fattore di penalizzazione sufficientemente grande, scelto in modo tale che la soluzione ottimale è data da $\mathbf{u} = \underline{0}$. Questo è un modo più "*compatto*" per eseguire l'algoritmo del simplesso, senza dover separare la fase I dalla fase II.



Dualità Dei Problemi di Programmazione Lineare

X

Definizione dei problemi duali per problemi primali PL. Esempio introduttivo. Definizione di problema duale, tabelle di conversione. Derivazione del problema duale in termini di rilassamento del problema primale.

X

0. Voci correlate

- Formulazione di Problemi Programmazione Lineare

1. Esempio Introduttivo dei Problemi Duali

Sia il problema di ottimizzazione formulato come

$$\begin{aligned} z &= \max \mathbf{c}^T \mathbf{x} \\ \mathbb{A} \mathbf{x} &\leq \mathbf{b} \\ \mathbf{x} &\geq 0 \end{aligned}$$

In particolare, lo interpretiamo nella seguente ottica: un'azienda produce n **beni** usando m **materiali**. Dunque $\mathbb{A} \in \mathbb{R}^{m \times n}$, dove $b_i = 1, \dots, m$ è la **disponibilità** dell' m -esimo materiale, a_{ij} è la quantità necessaria dell' i -esimo **materiale** per produrre il **bene** j -esimo. Vogliamo massimizzare il profitto z , dato da c_i il costo del **bene** i -esimo. Dunque semplicemente si ha che x_i è la quantità del materiale i -esimo prodotto.

Cosa succede, se invece di **utilizzare le risorse** per produrre i **beni**, li vendessi **direttamente**? Definiamo dunque le variabili decisionali $\underline{\pi} \in \mathbb{R}^m$ con π_i **prezzo** imposto per il materiale i -esimo. L'obiettivo funzione è definito dalla **vendita** di tutte le risorse disponibili per il loro prezzo, ossia il prodotto

$$\mathbf{b}^T \underline{\pi} = b_1 \pi_1 + \dots + b_m \pi_m$$

Siccome voglio che il **ricavo** sia maggiore rispetto alla **vendita "regolare"** dei beni, impongo la funzione obiettivo

$$\mathbb{A}^T \underline{\pi} \geq \mathbf{c}$$

Siccome voglio trovare il prezzo più **"conveniente"** da vendere, vado a minimizzare la funzione obiettivo. Quindi in definitiva il problema di ottimizzazione diventa

$$\begin{aligned} \omega &= \min \mathbf{b}^T \underline{\pi} \\ \mathbb{A}^T \underline{\pi} &\geq \mathbf{c} \\ \underline{\pi} &\geq 0 \end{aligned}$$

Notiamo che:

- Ogni soluzione ammissibile del problema duale ci consente di trarre un **profitto** che non sia più basso dato da una soluzione del problema **primale** (ossia producendo al posto di vendendo).
- Ogni valore ottimo del duale è sempre $\geq$ al valore ottimo del primale, sennò i vincoli non sarebbero rispettati

Vedremo, in seguito, che ci sono delle relazioni più profonde tra i valori e le soluzioni ottime del problema primale e duale.

Osserviamo che un **problema duale** di un **problema primale** non è altro quando i vettori $\mathbf{b}, \mathbf{c}$ si scambiano i ruoli e la matrice dei coefficienti $\mathbb{A}$ viene trasposta.

2. Rilassamento Lagrangiano della Primale

Q. Abbiamo visto che le variabili $\geq$ diventano dei vincoli $\leq$ nel problema duale, e viceversa con i vincoli del problema primale. Cosa succede, in presenza di casi misti ossia $=$ o $\leq$?

Una risposta viene fornita dall'interpretazione della dualità in termini di *rilassamento lagrangiano* del problema primale.

Consideriamo un problema di minimizzazione in forma standard:

$$\begin{aligned} z &= \min \mathbf{c}^T \mathbf{x} \\ \mathbb{A} \mathbf{x} &= \mathbf{b} \\ \mathbf{x} &\geq 0 \end{aligned}$$

Cosa succede se voglio *togliere i vincoli* $\mathbb{A} \mathbf{x} = \mathbf{b}$? Possiamo farlo modificando la funzione obbiettivo, rimpiazzando effettivamente $\mathbb{A} \mathbf{x} = \mathbf{b}$ con la "*penalizzazione*" $\mathbf{p}^T (\mathbf{b} - \mathbb{A} \mathbf{x})$, dove $\mathbf{p}$ è un vettore che ha le stesse dimensioni di $\mathbf{b}$.

$$\begin{aligned} \hat{z} &= \min \mathbf{c}^T \mathbf{x} + \mathbf{p}^T (\mathbf{b} - \mathbb{A} \mathbf{x}) \\ \mathbf{x} &\geq 0 \end{aligned}$$

Sicuramente, il valore ottimale del problema rilassato è *migliore* del problema originale, in quanto stiamo rilassando il problema. Ovvero, $\forall \mathbf{p}, \hat{z} \leq z^*$. Infatti il termine di penalizzazione verrebbe cancellato in quanto $\mathbb{A} \mathbf{x}^* = \mathbf{b}$.

Fissiamo $\mathbf{p}$ e definiamo il valore ottimo del problema rilassato come $g_{\mathbf{p}}$, ovvero

$$g_{\mathbf{p}} := \min_{\mathbf{x} \geq 0} [\mathbf{c}^T \mathbf{x} + \mathbf{p}^T (\mathbf{b} - \mathbb{A} \mathbf{x})]$$

Per trovare il miglior valore del problema rilassato, vogliamo che questa "*cresca*" il più possibile vicino al problema originale. Pertanto vogliamo ottimizzare il problema

$$\max_{\mathbf{p}} g_{\mathbf{p}}$$

Adesso vediamo che questa non è altro che la formulazione del *problema duale*. Effettuiamo i passaggi matematici per dimostrarlo:

$$\max_{\mathbf{p}} g_{\mathbf{p}} = \max_{\mathbf{p}} \left(\min_{\mathbf{x} \geq 0} [\mathbf{c}^T \mathbf{x} + \mathbf{p}^T (\mathbf{b} - \mathbb{A} \mathbf{x})] \right)$$

Espando il termine di penalizzazione

$$\begin{aligned} \max_{\mathbf{p}} \left(\min_{\mathbf{x} \geq 0} [\mathbf{c}^T \mathbf{x} + \mathbf{p}^T (\mathbf{b} - \mathbb{A} \mathbf{x})] \right) &= \max_{\mathbf{p}} \left(\min_{\mathbf{x} \geq 0} [\mathbf{c}^T \mathbf{x} + \mathbf{p}^T \mathbf{b} - \mathbf{p}^T \mathbb{A} \mathbf{x}] \right) \\ &= \max_{\mathbf{p}} \mathbf{p}^T \mathbf{b} + \left(\min_{\mathbf{x} \geq 0} [\mathbf{c}^T \mathbf{x} - \mathbf{p}^T \mathbb{A} \mathbf{x}] \right) \\ &= \max_{\mathbf{p}} \mathbf{p}^T \mathbf{b} + \left(\min_{\mathbf{x} \geq 0} [(\mathbf{c}^T - \mathbf{p}^T \mathbb{A}) \mathbf{x}] \right) \end{aligned}$$

Il secondo termine è minimo in due casi:

- Se $\mathbf{c}^T - \mathbf{p}^T \mathbb{A} \geq 0$, allora sicuramente $\mathbf{x}^* = 0$
- Altrimenti $\mathbf{x}^* = -\infty$.

Il secondo caso non ci porterà sicuramente al del problema, quindi posso porre $\mathbf{c}^T - \mathbf{p}^T \mathbb{A} \geq 0$ come un *vincolo* e otterrò massimo

$$\mathbf{p}^T \mathbb{A} \leq \mathbf{c}^T$$

E posso *"cancellarlo"* dalla penalità (siccome $\mathbf{x}^* = 0$), ottenendo finalmente il problema

$$\omega = \max_{\mathbf{p}^T \mathbb{A} \leq \mathbf{c}^T} \mathbf{p}^T \mathbf{b}$$

che è proprio il problema *duale* del *primale*.

Osserviamo che se avessimo un vincolo del tipo $\mathbb{A}\mathbf{x} \geq \mathbf{b}$, è sufficiente introdurre el *variabili di surplus* $\mathbb{A}\mathbf{x} - \mathbf{s} = \mathbf{b}, \mathbf{s} \geq 0$ e rifare l'intero procedimento considerando che

$$\mathbb{A}\mathbf{x} - \mathbf{s} = \mathbb{A}\mathbf{x} - \mathbb{1}\mathbf{s} = (\mathbb{A} | -\mathbb{1}) \begin{pmatrix} \mathbf{x} \\ \mathbf{s} \end{pmatrix} = \mathbf{b}$$

Infatti otterrei i *vincoli duali*

$$\mathbf{p}^T (\mathbb{A} | -\mathbb{1}) \leq (\mathbf{c}^T | 0^T)$$

Ossia

$$\mathbf{p}^T \mathbb{A} \leq \mathbf{c}^T \wedge \mathbf{p} \geq 0$$

Se, modificando le condizioni sulle variabili, $\mathbf{x}$ è *"libero"* (ovvero non soggetto a vincoli di segno), usando un ragionamento analogo otteniamo

$$\min_{\mathbf{x} \text{ free}} (\mathbf{c}^T - \mathbf{p}^T \mathbb{A})\mathbf{x} = \begin{cases} 0, & \mathbf{c}^T - \mathbf{p}^T \mathbb{A} = 0 \\ -\infty, & \text{alt.} \end{cases}$$

Infatti se l'iperpiano non fosse *"piatto"*, allora sarebbe coercitiva in $\mathbf{x}$ libera e dunque sarebbe illimitata. Dunque poniamo

$$\mathbf{p}^T \mathbb{A} = \mathbf{c}^T$$

Queste considerazioni andranno a motivare la seguente *"tabella della conversione"* tra il problema primale e duale.

PRIMALE	max	min	DUALE
	$\leq b_i$	≥ 0	
Vincolo i -esimo	$\geq b_i$	≤ 0	Variabile i -esimo
	$= b_i$	free	
	≥ 0	$\geq c_j$	
Variabile j -esimo	≤ 0	$\leq c_j$	Vincolo j -esimo
	free	$= c_j$	

In altre parole, i vincoli $\leq$ diventano variabili ≥ 0 (o libere se $=$) e per le variabili non si cambia segno (a meno di variabili libere, allora si avrebbe l'uguaglianza stretta).

Esempio.

$$\begin{aligned} z &= \max 2x_1 + 3x_2 - x_3 + 7x_4 \\ 4x_1 + 3x_2 - x_3 + 2x_4 &\leq 35 \\ x_1 + 5x_2 + 6x_3 + 10x_4 &= 28 \\ 2x_1 + 7x_2 - 2x_3 + 4x_4 &\geq 15 \\ x_1 &\geq 0, x_2 \in \mathbb{R}, x_3 \leq 0, x_4 \geq 0 \end{aligned}$$

Identifichiamo la matrice $\mathbb{A}$ come

$$\mathbb{A} = \begin{pmatrix} 4 & 3 & -1 & 2 \\ 1 & 5 & 6 & 10 \\ 2 & 7 & -2 & 4 \end{pmatrix}$$

Ovviamente la sua trasposta è

$$\mathbb{A}^T = \begin{pmatrix} 4 & 1 & 2 \\ 3 & 5 & 7 \\ -1 & 6 & -2 \\ 2 & 10 & 4 \end{pmatrix}$$

Considerando la tabella delle conversioni, ottengo il problema duale

$$\begin{aligned} \omega &= \min 35\pi_1 + 28\pi_2 + 15\pi_3 \\ 4\pi_1 + 1\pi_2 + 2\pi_3 &\geq 2 \\ 3\pi_1 + 5\pi_2 + 7\pi_3 &= 3 \\ -1\pi_1 + 6\pi_2 - 2\pi_3 &\leq -1 \\ 2\pi_1 + 10\pi_2 + 4\pi_3 &\geq 7 \\ \pi_1 &\geq 0, \pi_2 \in \mathbb{R}, \pi_3 \leq 0 \end{aligned}$$

Legami tra i Problemi Primali e Duali

X

Teoremi sui problemi duali e primali: dualità debole e forte, corollari. Implicazione degli esiti tra PL e DPL. Complementarietà dello scarto.

X

0. Voci correlate

- [Dualità Dei Problemi di Programmazione Lineare](#)

1. Legami tra gli Esiti

Q. Dato un problema primale PL e il suo problema **duale** DPL, quali relazioni ci sono tra le loro *soluzioni* e *valori ottimi*?

Il punto cruciale della *dualità* sono proprio i teoremi su questo aspetto.

#Teorema

Teorema (della dualità debole).

Sia PL un problema primale (con le variabili x) e DPL il suo problema duale (con le variabili π). Assumiamo WLOG che PL è un problema di massimizzazione.

Se x, π sono soluzioni ammissibili, allora vale la maggiorazione

$$c^T x \leq b^T \pi$$

Ovvero il valore fornito dalla soluzione π è un upper bound di tutti i valori ottimi della PL.

#Dimostrazione

DIMOSTRAZIONE del Teorema 1

Notiamo che $A^T \pi \geq c$, per cui

$$c^T x \leq (A^T \pi)^T x$$

La trasposta di una moltiplicazione tra matrice e vettore è la trasposta distribuita con ordine invertito:

$$(A^T \pi)^T x = \pi^T A x$$

Per il problema primale vale che $Ax \leq b$, da cui

$$\pi^T A x \leq \pi^T b$$

che è la tesi. Il fatto che si può assumere che il problema primale è un problema di massimizzazione senza perdere di generalità deriva dal fatto che si può assumere il duale come la primale, e procedere con la stessa maniera. ■

#Corollario

Corollario (relazioni tra problemi primali e duali in caso di non-esistenza della soluzione).

Sia PL il problema primale, e DPL il suo problema duale. Sia PL un problema di massimizzazione WLOG.

Allora vale che:

Se il valore ottimo di PL è illimitato, allora il problema duale è inammissibile

Se il valore ottimo di DPL è illimitato, allora il problema primale è inammissibile

#Dimostrazione

DIMOSTRAZIONE del Corollario 2

Dimostriamo il primo punto. Se il valore ottimo di PL è illimitato, allora $c^T x = +\infty$. Tuttavia per la dualità debole vale che

$$+\infty \leq b^T \pi$$

Che vale per $b = +\infty$, che è impossibile (pensa di definire una regione geometrica "lontanissima"...). Il secondo punto si dimostra analogamente, basandosi sulla dualità debole. ■

#Corollario

Corollario (uguaglianza dei valori ottimi implica l'ottimalità delle soluzioni).

Sia PL il problema PL di massimizzazione (WLOG) e DPL il suo problema duale.

Se x, π sono soluzioni ammissibili a PL, DPL tali che $c^T x = b^T \pi$, allora vale che x, π sono le *soluzioni ottime* ai problemi PL, DPL.

#Dimostrazione

DIMOSTRAZIONE del Corollario 3

Per la dualità debole ho che

$$c^T y \leq c^T x = b^T \pi$$

Allora ho che $c^T y \leq c^T x, \forall y$ per cui la tesi. ■

#Teorema

Teorema (dualità forte).

Se un problema PL ha una *soluzione ottima*, allora lo ha pure il suo problema duale *DPL* e vale che i loro valori ottimi sono uguali.

Ossia, se esiste $x^* : z = \max c^T x^*$, allora vale che $\exists \pi^* : \omega = \min b^T \pi$ e che $c^T x^* = b^T \pi^*$.

La dimostrazione è lunga e dunque omessa.

Dunque, possiamo riassumere tutti i teoremi che vanno a descrivere la relazione tra gli *esiti* di una PL con la sua DPL col seguente teorema.

Teorema (relazione tra problemi primali e duali).

Sia PL un problema di ottimizzazione e DPL il suo duale. Allora vale che:

- PL ha una soluzione ottima se e solo se DPL ha una soluzione ottima
- Se PL è illimitata allora DPL è inammissibile (e viceversa, scambiando i ruoli)
- Se PL è inammissibile, allora DPL o ha soluzioni infinite o è inammissibile (WLOG)

In altre parole, ho la seguente tabella:

PL => DPL ?	Ottimo Finito	Illimitato	Innammissibile
Ottimo Finito	Sì	No	No
Illimitato	No	No	Sì
Innammissibile	No	Sì	Sì

#Dimostrazione

DIMOSTRAZIONE del $\wedge d1a66a$

I primi due punti sono già dimostrati. Dimostriamo solo un esempio in cui un problema primale PL e il suo duale DPL sono inammissibili allo stesso tempo.

Consideriamo il primale inammissibile

$$\begin{aligned}
 z &= \max x_1 + x_2 \\
 x_1 + x_2 &= 1 \\
 2x_1 + 2x_2 &= 3 \\
 x_1, x_2 &\in \mathbb{R}
 \end{aligned}$$

Ovviamente questa è inammissibile. Prendendo la sua duale ho

$$\begin{aligned}
 \omega &= \max \pi_1 + 3\pi_2 \\
 \pi_1 + 2\pi_2 &= 1 \\
 \pi_2 + 2\pi_2 &= 1 \\
 \pi_1, \pi_2 &\in \mathbb{R}
 \end{aligned}$$

Che è inammissibile, concludendo l'esempio. ■

2. Scarto Complementare

Adesso vediamo come si comportano le soluzioni dei problemi duali, tra di loro.

#Teorema

Teorema (lo scarto complementare).

Sia PL un problema di ottimizzazione (massimizzazione) e DPL il suo problema duale.

Allora, dati x, π delle soluzioni ammissibili per PL, DPL, si ha che sono delle *soluzioni ottime* se e solo se vale che

$$\begin{aligned}\forall i, \pi_i(b_i - A_{(i)}^T x) &= 0 \\ \forall j, (\pi^T A_{(j)} - c_j) &= 0\end{aligned}$$

Ossia

$$\pi(b - A^T x) = (\pi^T A - c)x = \underline{0}$$

#Dimostrazione

DIMOSTRAZIONE del Teorema 6

Omessa. ■

Questo teorema è utile per dedurre la soluzione in un problema duale a partire dalla soluzione del problema primale.

Consideriamo il PL

$$\begin{aligned}z &= \max 3x_1 + 5x_2 \\ x_1 + 0x_2 &\leq 4 \\ 0x_1 + 2x_2 &\leq 12 \\ 3x_1 + 2x_2 &\leq 18 \\ x_1, x_2 &\geq 0\end{aligned}$$

Allora il suo duale è

$$\begin{aligned}\omega &= \min 4\pi_1 + 12\pi_2 + 18\pi_3 \\ \pi_1 + 3\pi_3 &\geq 3 \\ 2\pi_2 + 2\pi_3 &\geq 5 \\ \pi_1, \pi_2, \pi_3 &\geq 0\end{aligned}$$

Consideriamo il problema primale PL. Allora, scegliendo $i = 1$ ho che

$$\pi_1(4 - x_1) = 0$$

Analogamente per $i = 2, 3$ ottengo

$$\begin{aligned}\pi_2(12 - 2x_2) &= 0 \\ \pi_3(18 - 3x_1 - 2x_2) &= 0\end{aligned}$$

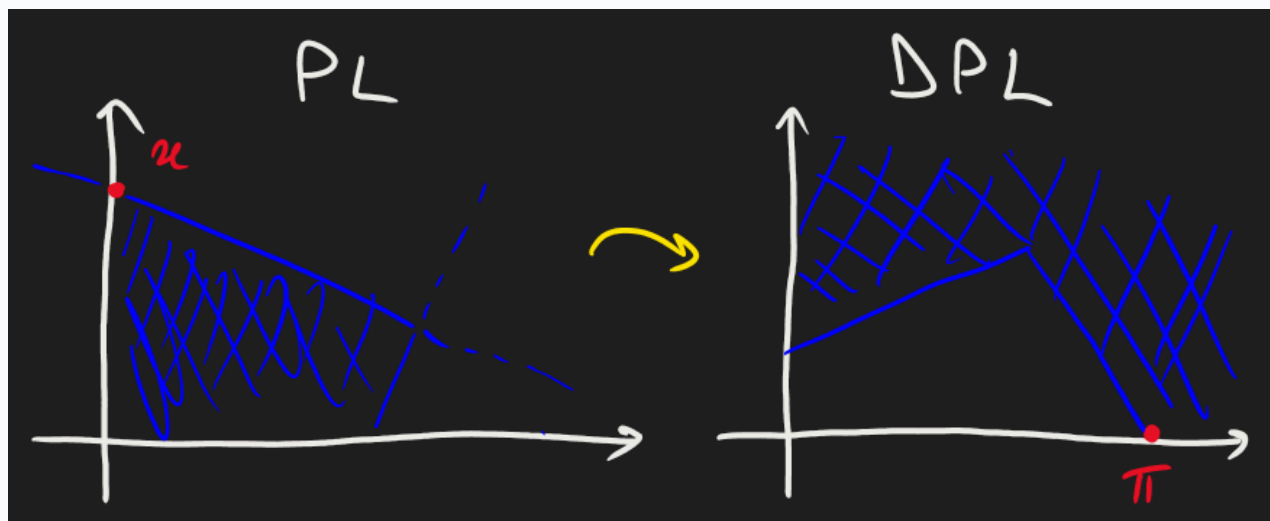
Introduciamo le variabili slack, ossia (x_1, x_2, x_3) sono date da

$$\begin{cases} 4 - x_1 = s_1 \\ 12 - 2x_2 = s_2 \\ 18 - 3x_1 - 2x_2 = s_3 \end{cases}$$

Pertanto le equazioni date dal teorema dello scarto complementare ho

$$\underline{\pi}^T \underline{s} = 0$$

Questa è vera *se e solo se* per ogni componente i -esima, π_i o s_i è nulla (e che $\underline{x}, \underline{\pi}$ siano ottime). Questo vuol dire che se nel problema primale mi trovo su un vincolo, allora nel problema duale sono su una *componente nulla*; viceversa, se nel problema duale non ho la componente nulla, allora devo trovarmi su un vincolo in quanto s_i è nulla. Si può effettuare un ragionamento analogo con le *variabili di surplus* nel problema duale DPL. ■



Concludiamo osservando che:

- Se vale " $\Leftarrow$ " (ovvero che ho *due soluzioni di base* che soddisfano le equazioni) ma non " $\Rightarrow$ " (ovvero che le soluzioni sono entrambi *ottime*), allora vuol dire che *non ho l'ammissibilità* delle soluzioni. Infatti, sono entrambi ammissibili *se e solo se* sono entrambi ottimi.
- Possiamo riformulare in termini di vincoli *stretti* (ossia $<$ e $=$).

Analisi di Sensibilità dei PL

X

Analisi di sensibilità dei PL. Definizione e obiettivi di analisi di sensibilità. Definizione di prezzo ombra. Esempi e osservazioni. Il grafico dei prezzi ombra rispetto ai coefficienti vincolari è una funzione convessa. Analisi di sensibilità rispetto ai coefficienti della funzione obiettivo.

0. Voci correlate

- Formulazione di Problemi Programmazione Lineare
- Dualità Dei Problemi di Programmazione Lineare

1. Idea dell'Analisi di Sensibilità

Q. Supponiamo di avere un problema PL con i coefficienti (c, A, b) , conoscendo il suo valore ottimale z e la sua soluzione x^* . Cosa succede a x^* e z se facciamo "variare" leggermente i parametri $(c, A, b) \rightsquigarrow (c', A', b')$, senza dover rifare i conti?

Questa è proprio l'obiettivo *principale* dell'analisi di sensibilità (intesa nel senso di saggezza, sensatezza...). In particolare, vogliamo identificare i *parametri sensibili*, ovvero che facendoli variare cambia drasticamente la soluzione ottimale. Nella realtà, questa è utile per minimizzare i *rischi* di ottenere soluzioni ottimali erranee.

In particolare, quando parliamo di *analisi di sensibilità* ci troviamo nel seguente framework:

- I problemi PL sono interpretati come problemi di "allocazione delle risorse a più attività"
- Di solito i vincoli sono di forma " $\leq$ ", ossia il numero di risorse disponibili per una certa attività
- I parametri b_i sono dei "valori iniziali"
Quindi, trovando i parametri sensibili e non sensibili b_i , possiamo revisionare il nostro modello PL per ottenere *esiti migliori*.

2. Sensibilità Rispetto ai Vincoli

#Definizione

Definizione (prezzo ombra rispetto ad una risorsa).

Sia (c, A, b) un PL. Definiamo il *prezzo ombra* rispetto al *coefficiente vincolare* b_i come la *variazione* del valore ottimale per "*variazioni piccole*" di b_i . La denotiamo con y_i^* .

Notiamo che se il vincolo è di forma $=$ o $\geq$, la definizione rimane uguale ma è da interpretare in termini diversi.

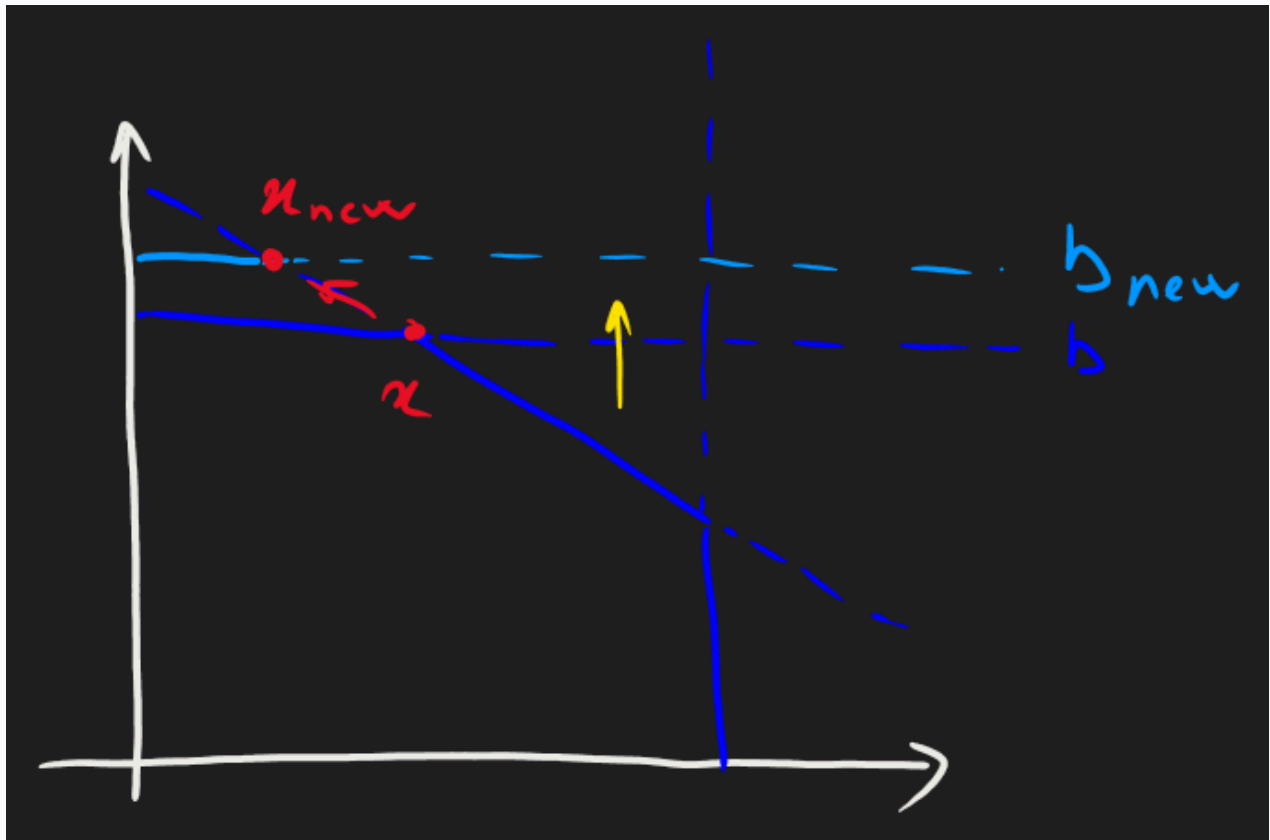
Facciamo un *esempio* di *prezzo ombra*. Sia PL il seguente problema:

$$\begin{aligned}
z &= \max 3x_1 + 5x_2 \\
1x_1 + 0x_2 &\leq 4 \\
0x_1 + 2x_2 &\leq 12 \\
3x_1 + 2x_2 &\leq 18 \\
x_1, x_2 &\geq 0
\end{aligned}$$

Risolvendolo con l'algoritmo del simplesso si avrebbe che la soluzione giace in $x^* = (2, 6)$, con $z = 36$. Avendo $b = (4, 12, 18)$. Cosa succede se facciamo "*variare*" b_2 , cambiando $12 \rightarrow 13$? Vediamo che otteniamo la nuova soluzione $x_{\text{new}}^* = (5/3, 13/2)$, ottenendo dunque il nuovo valore ottimale

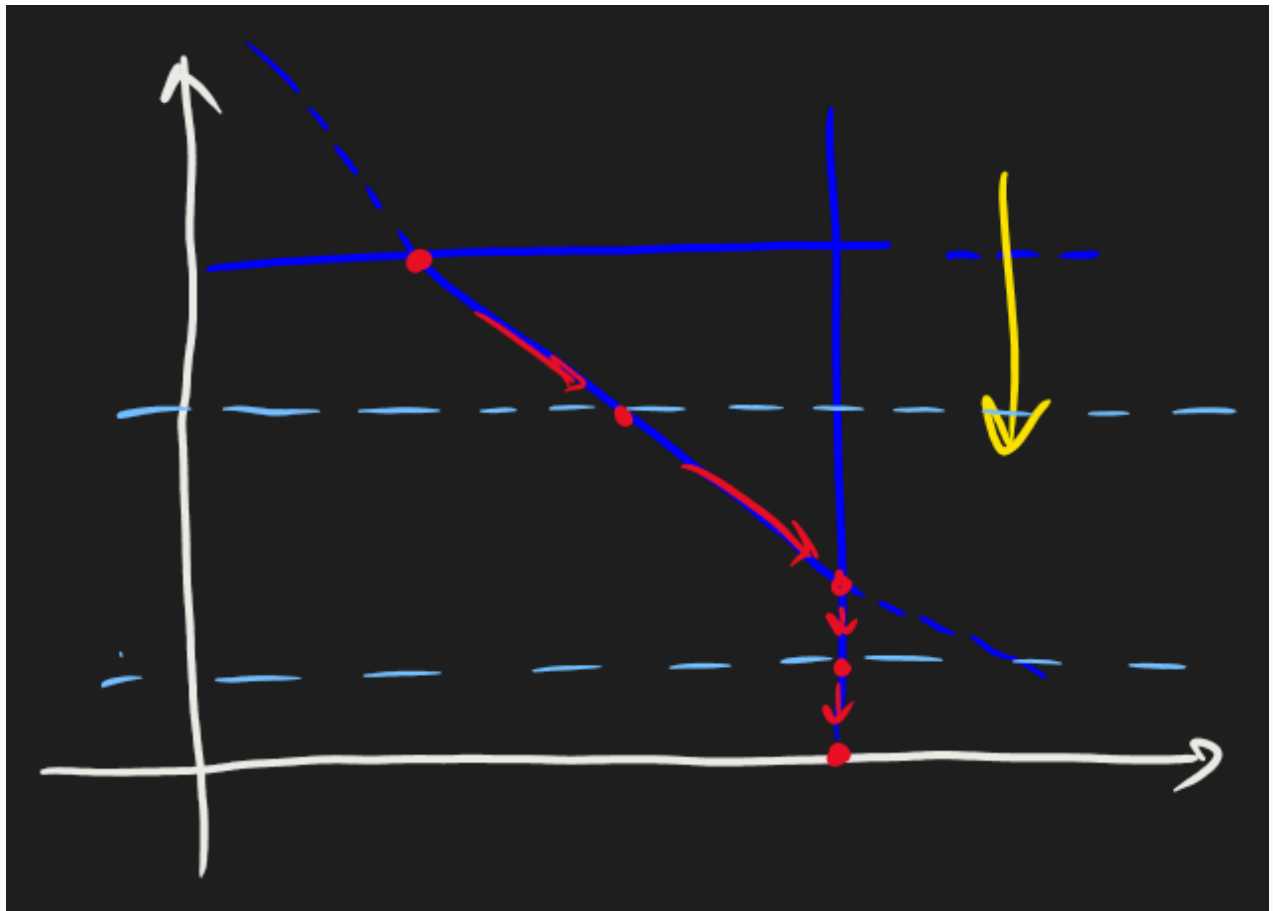
$$z_{\text{new}} = 3(5/3) + 5(13/2) = 37 + 1/2$$

Pertanto il prezzo ombra è $y_2^* = \Delta z = z_{\text{new}} - z = 3/2$. Geometricamente, stiamo facendo muovere il "*secondo vincolo*" (ovvero la retta orizzontale) sopra, spostando dunque "*in sù*" la soluzione.



Notiamo tuttavia, da questa intuizione geometrica, che ad un certo punto il prezzo ombra smette di variare. Geometricamente, spostando ulteriormente b_2 avrei che la soluzione ottimale non può più spostarsi in quanto raggiunge il limite $x = 0$. Formalmente, diciamo che il *vincolo* $2x_2 = b_2$ diventa *ridondante* e dunque il prezzo ombra diventa zero.

Abbiamo più o meno lo stesso discorso se *diminuisco* b_2 , ossia faccio "*spostare*" la soluzione ottimale finché non raggiunga la non ammissibilità $2x_2 < 0$. Tuttavia notiamo che ad un certo punto ($0 \leq b_2 \leq 4$) la soluzione smette di "*muoversi*" sul terzo vincolo (ossia la retta s_3), inizia a spostarsi sulla *linea verticale* s_1 .

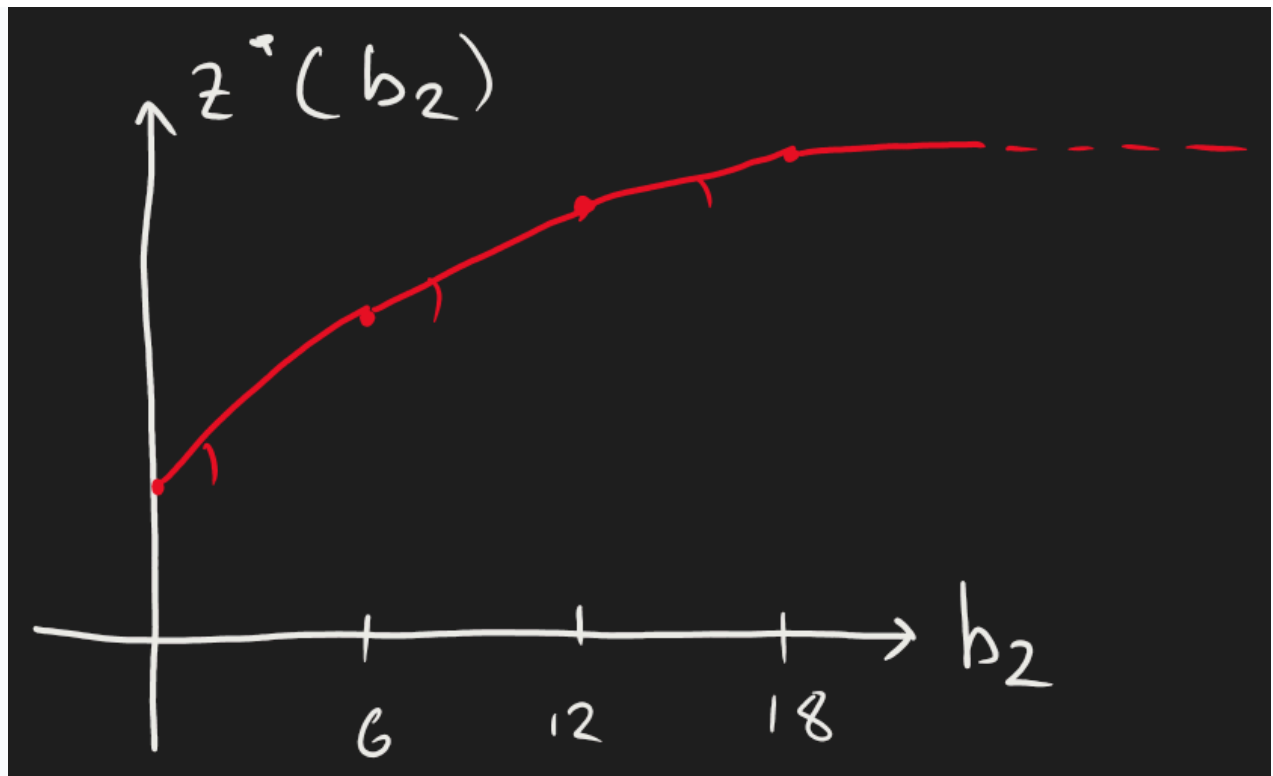


Cosa succede in termini matematici? Sto cambiando le *basi* delle soluzioni, in effetti se prima le variabili *fuori base* (ossia nulli) erano s_2, s_3 , dopo le variabili *fuori base* diventano s_1, s_2 .

Ripetendo lo stesso ragionamento per b_1, b_2 , ottenendo dunque lo schema riepilogativo

$$\begin{aligned} y_1^* &= 0 \\ y_2^* &= 1.5 \\ y_3^* &= 1 \end{aligned}$$

Concludiamo osservando che tracciando il grafico di z^* rispetto a b_2 , abbiamo una *funzione convessa* (e lineare a tratti). ■



Adesso osserviamo che il *problema duale* può aiutarci a ricavare i valori y_i^* .

Infatti il suo problema duale è

$$\begin{aligned}\omega &= \min 4\pi_1 + 12\pi_2 + 18\pi_3 \\ 1\pi_1 + 0\pi_2 + 3\pi_3 &\geq 3 \\ 0\pi_1 + 2\pi_2 + 2\pi_3 &\geq 5 \\ \pi_1, \pi_2, \pi_3 &\geq 0\end{aligned}$$

Con le variabili di surplus ho i vincoli riscritti come

$$\begin{aligned}1\pi_1 + 0\pi_2 + 3\pi_3 - r_1 &= 3 \\ 0\pi_1 + 2\pi_2 + 2\pi_3 - r_2 &= 5\end{aligned}$$

Sapendo che x^* giace sul secondo e terzo vincolo (ossia $s_2, s_3 = 0, s_1 > 0$) ottengo, per la *complementarietà dello scarto*, che $\pi_1^* = 0$ e $r_1^*, r_2^* = 0$ ($x_1^*, x_2^* > 0$). Pertanto le equazioni diventano

$$\begin{aligned}3\pi_3 &= 3 \\ 2\pi_2 + 2\pi_3 &= 5 \iff 2\pi_2 = 3\end{aligned}$$

Pertanto la soluzione ottimale duale è

$$\pi_1^* = 0, \pi_2^* = \frac{3}{2}, \pi_3^* = 1$$

Enunciamo dunque il seguente risultato:

#Teorema

Teorema (prezzo ombra e soluzioni della duale).

Sia PL un problema di ottimizzazione (di programmazione lineare), e sia DPL la sua duale. Se PL ammette una soluzione ottima, allora il *prezzo ombra* dei coefficienti b_i è data dalla soluzione ottima della DPL π_i^* . Ossia

$$\forall i, \pi_i^* = y_i^*$$

Osserviamo che al variare di b_i la soluzione ottima rimane uguale. Infatti, cambiano *solo* i coefficienti della funzione obiettivo di DPL. Infatti le funzioni obiettivo si coincidono:

$$z^* = \omega^* \implies 3x_1 + 5x_2 = 4\pi_1 + b_2\pi_2 + 8\pi_3$$

X

3. Sensibilità Rispetto ai Coefficienti

Q. Cosa succede se, invece di cambiare i coefficienti $b \rightsquigarrow b'$, cambiassimo i coefficienti della funzione obiettivo $c \rightsquigarrow c'$?

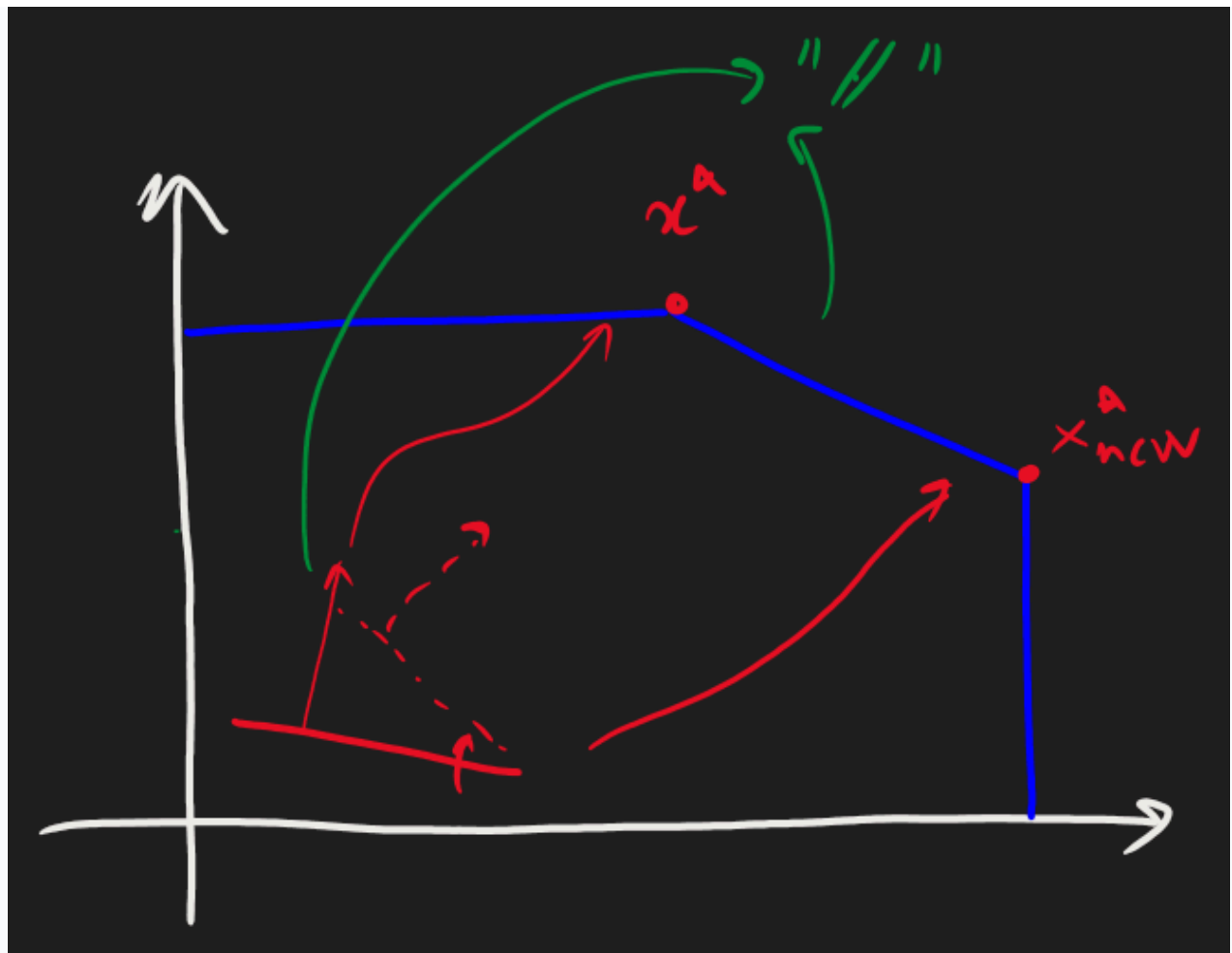
Facendo un'osservazione geometrica, notiamo che *cambiare "leggermente"* i coefficienti c corrisponde a *"ruotare leggermente"* l'iperpiano descritto da $c^T x$. Prendiamo il caso due-dimensionale, con la seguente funzione obiettivo:

$$\begin{aligned} z = \max \quad & 3x_1 + 5x_2 \\ & 1x_1 + 0x_2 \leq 4 \\ & 0x_1 + 2x_2 \leq 12 \\ & 3x_1 + 2x_2 \leq 18 \\ & x_1, x_2 \geq 0 \end{aligned}$$

Ovvero lo stesso problema della sezione precedente. Aumentando man mano c_1 , potrebbe verificarsi c diventi *parallela* ad un vincolo. In particolare al vincolo $3x_1 + 2x_2$; risolvendo l'equazione

$$\hat{c} : 5 = 3 : 2$$

Otteniamo che per $\hat{c} = \frac{15}{2}$, l'iperpiano descritto dalla funzione obiettivo diventa *parallelo* al terzo vincolo. Pertanto avremmo lo stesso valore ottimo, ma la soluzione x^* non è più unica.



Pertanto se *"spingessimo"* ulteriormente la rotazione (ossia ponendo $c_{1,\text{new}} > \frac{15}{2}$), allora la soluzione *non* si troverà più sul terzo vincolo, bensì si troverà sul secondo vincolo (stiamo in effetti "pivotando" tra il secondo e terzo vincolo). Il discorso è analogo per $c_{1,\text{new}} = 0$: la funzione obbiettivo diverrebbe parallela al primo vincolo, cambiando dunque soluzione.

Si conclude definendo un *"rango accettabile"* come l'intervallo per cui c ritiene le stesse soluzioni.

$$C_1 = \left[0, \frac{15}{2}\right]$$

"Programmazione Intera"

Problema del Commesso Viaggiatore

X

Problema del commesso viaggiatore. Formulazione del problema in termini di programmazione intera.

X

0. Voci correlate

1. Descrizione del Problema

PROBLEMA. Sia $G = (V, E)$ un grafo direzionato. Ossia, $(i, j) \in E$ è una coppia. Supponiamo inoltre, che ad ogni arco associamo il "*peso*" $w_{ij} : (i, j) \in E \mapsto \mathbb{R}^+$. Per semplicità supponiamo che $w_{ij} = w_{ji}$. Allora il *problema del commesso viaggiatore* va a *identificare* un *cammino Hamiltoniano* (ovvero che va a visitare tutti i nodi solamente una volta) col *costo minimo*.

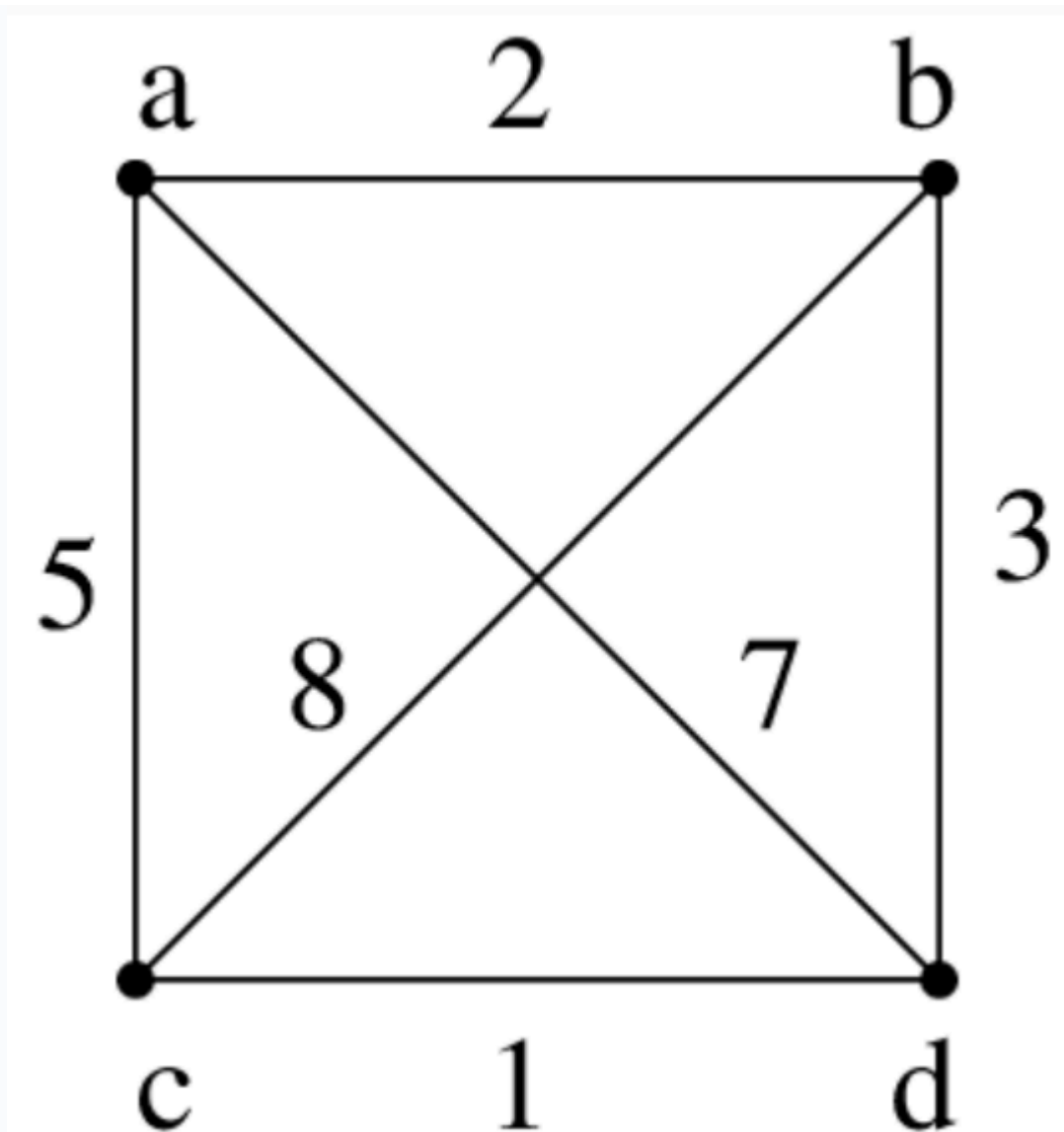
Questo problema potrebbe essere inizialmente risolto *cercando* tutti i *cammini Hamiltoniani* e scegliere quella migliore. Nei casi più semplici è possibile prendere questa strategia, come definendo

$$\begin{aligned} V &= \{a, b, c, d\} \\ E &= (V \times V) \setminus \{a \in V : (a, a)\} \\ w_{ab} &= 2, w_{ac} = 5, w_{ad} = 7 \\ w_{bc} &= 8, w_{bd} = 3 \\ w_{cd} &= 1 \end{aligned}$$

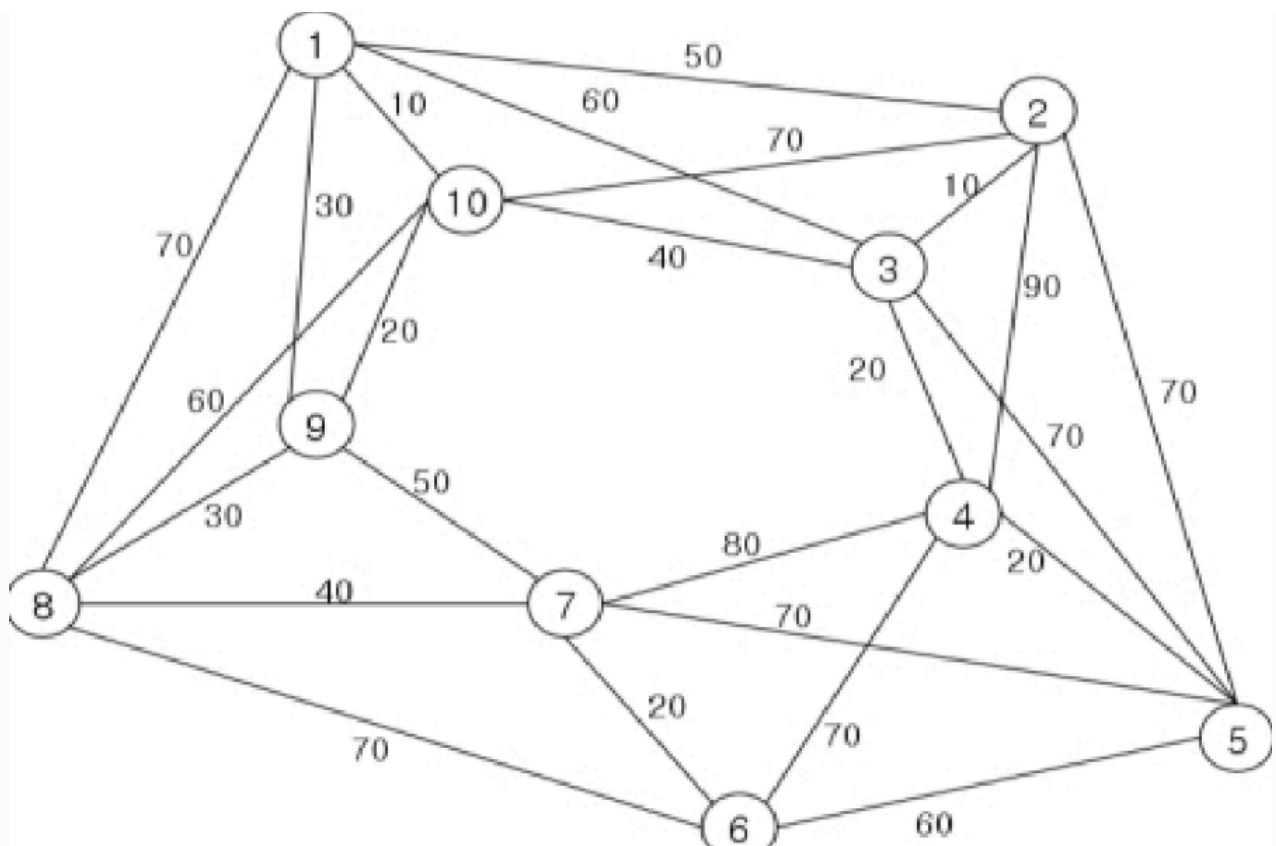
Allora avrei solamente tre cammini Hamiltoniani, identificati come:

$$\begin{aligned} \text{HAM}_1 &= (a, b, c, d, c, a) \rightsquigarrow w_{\text{HAM}_1} = 11 \\ \text{HAM}_2 &= (a, d, b, c, a) \rightsquigarrow w_{\text{HAM}_2} = 23 \\ \text{HAM}_3 &= (a, d, c, b, a) \rightsquigarrow w_{\text{HAM}_3} = 18 \end{aligned}$$

Chiaramente il *primo* cammino Hamiltoniano è quello migliore.



Tuttavia, il problema diventa più difficile per un numero crescente di *nodi* e per una struttura "geometrica" più *complessa*. Infatti, il numero di cicli hamiltoniani disponibili è *maggiorata* dall'enumerazione dei nodi.



Vogliamo dunque trovare un modo più *"intelligente"* per risolvere il problema.

X

2. Formulazione del Commesso Viaggiatore in IP

Formuliamo il problema del *commesso viaggiatore* in termini di un problema di *programmazione intera*.

Variabili Decisionali: Come variabili decisionali definiamo x_{ij} dei valori booleani, che decidono se *"j segue i nel cammino"*. Pertanto, si ha che x è una matrice con dimensione $|E|$.

Funzione Obbiettivo: Come funzione obbiettivo decidiamo naturalmente di *minimizzare* il costo totale del percorso scelto.

$$\omega = \min \sum_{(i,j) \in E} c_{ij} x_{ij}$$

Vincoli: Per i vincoli abbiamo un discorso più complesso, in quanto voglio che il cammino rappresentato da x sia *Hamiltoniano*. Ossia, *tutte le città* sono entrate e uscite esattamente una volta. Vogliamo dunque le seguenti somme:

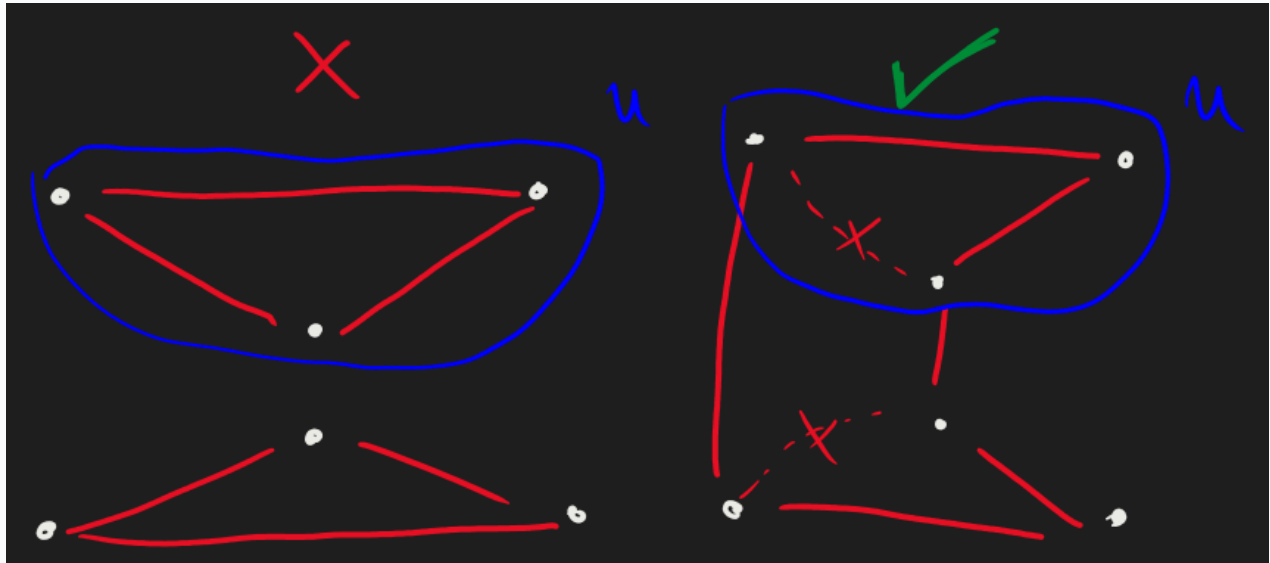
$$\sum x_{i\bullet} = \sum x_{\bullet j} = 1, \forall i, j \in V$$

Tuttavia, questo vincolo non è sufficiente. Infatti è possibile avere un cammino hamiltoniano che soddisfa l'equazione sopra. In particolare, quando abbiamo *due o più sottocicli* separati ma vanno comunque creare una situazione in cui tutti i punti sono entrati e usciti una volta.

Un modo per eliminare i sottocicli è quello di, dato un sottinsieme $U \subset V$, imporre che U e V abbiano *almeno un collegamento*. Ovvero abbiamo i $2 \leq |U| \leq |V| - 2$ vincoli:

$$\sum_{\{(i,j) \in E: i \in U, j \in V \setminus U\}} x_{ij} \geq 1$$

Tuttavia abbiamo una scelta arbitraria di U , pertanto dobbiamo tenere conto di *tutte le sue scelte possibili*, che sono al massimo $2^{|V|}$.



Un altro modo per eliminare i sottocicli è quello di richiedere per ogni sottinsieme U che ci siano *meno archi* dei *nodi* (in questo modo i sottoinsiemi non sono più "divisi"). Ovvero

$$\sum_{\{(i,j) \in E: i,j \in U\}} x_{ij} \leq |U| - 1$$

Però anche qui richiediamo approssimativamente $2^{|V|}$ vincoli, infatti il calcolo esatto è

$$\frac{1}{2} \left[\sum_{n=2}^{|V|-2} \binom{|V|}{n} \right] \approx 2^{|V|}$$

Approssimanti di Soluzioni IP

X

Relazioni tra problemi di programmazione lineare e intera. Osservazione iniziale: apparentemente le soluzioni PL sono lontane dalle soluzioni IP. Idea della ricerca dei legami: approssimanti dal basso e dall'alto del valore ottimale. Rilassamento continuo di un IP. Lower bound e upper bound di problemi IP.

X

0. Voci correlate

- Formulazione di Problemi Programmazione Lineare
- Problema del Commesso Viaggiatore

1. Problemi IP

Definiamo un *problema di programmazione intera* come il problema di ottimizzazione

$$\begin{aligned} Z = \max \quad & \underline{c}^T \underline{x} \\ & \underline{A}\underline{x} \leq \underline{b} \\ & \underline{x} \geq \underline{0} \wedge \underline{x} \in \mathbb{Z}^d \end{aligned}$$

Dove $\underline{c} \in \mathbb{R}^d$, $\underline{b} \in \mathbb{R}^c$ e dunque $\underline{A} \in \mathbb{R}^{c \times d}$.

Q. Conoscendo una suo problema "*corrispettivo*" in $\mathbb{R}^d$ (definiremo bene questa idea con la nozione di *rilassamento continuo*), posso far risalire le soluzioni del problema di programmazione intera?

Un primo approccio *naïve* è quello di, data una soluzione $\underline{x}^* \in \mathbb{R}^d$ quella di considerare il suo *arrotondamento* intero. Tuttavia, col seguente esempio dimostriamo che quest'idea non funziona.

Sia IP il seguente problema:

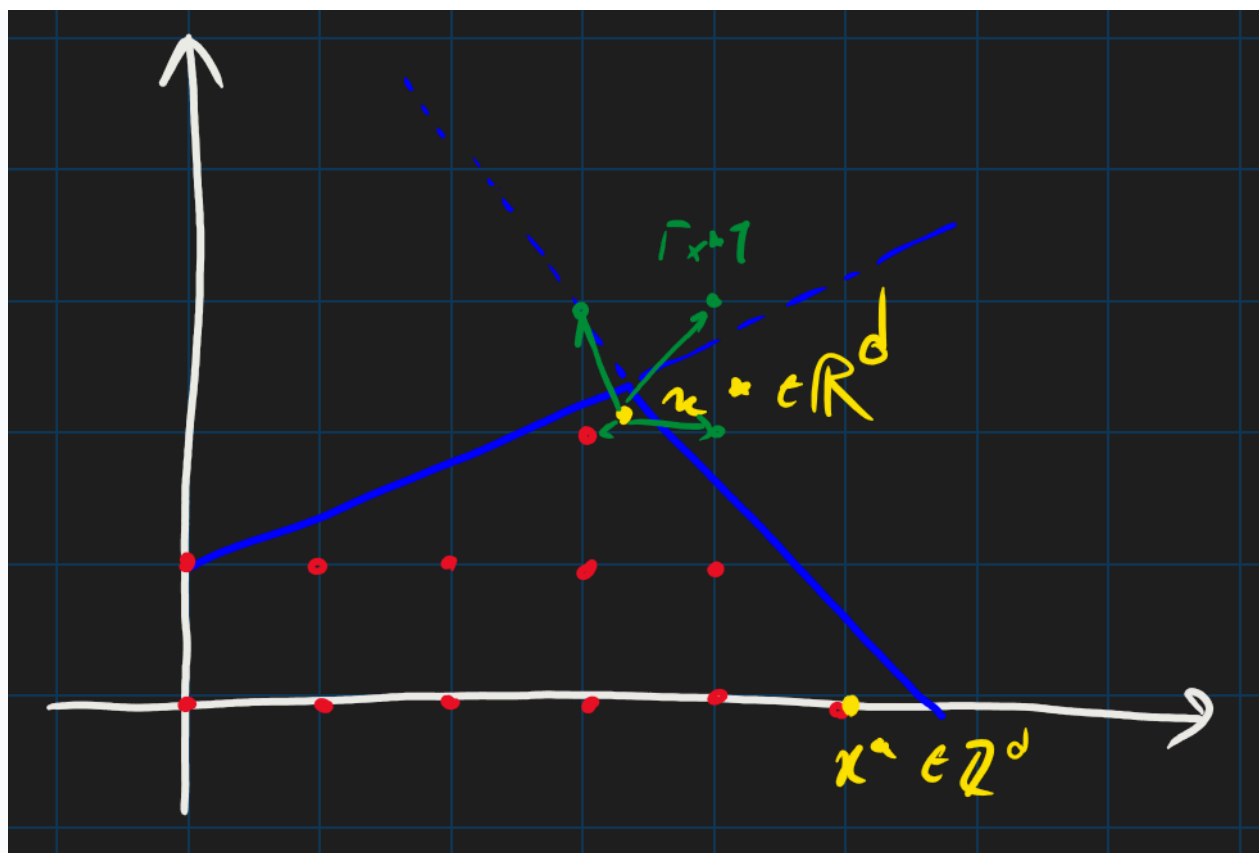
$$\begin{aligned} Z = \max \quad & 1.00x_1 + 0.64x_2 \\ & 50x_1 + 31x_2 \leq 250 \\ & 3x_1 - 2x_2 \geq -4 \\ & x_1, x_2 \geq 0, \in \mathbb{Z} \end{aligned}$$

La sua soluzione *intera* è $(5, 0)$, invece la sua soluzione *continua* è $(\frac{376}{193}, \frac{950}{193}) \approx (1.948, 4.922)$.

Un primo problema è quello di determinare *come* arrotondare la soluzione: notiamo che già qui non produciamo sempre delle *soluzioni ottimali*, infatti prendendo il *ceiling* $\lceil (1.948, 4.922) \rceil = (2, 5)$ scopriamo che questa non è ammissibile (infatti $50 \cdot 2 + 31 \cdot 5 = 255$).

Prendiamo invece il *floor* vettoriale, ovvero $\lfloor (1.948, 4.922) \rfloor = (1, 4)$. Come prima, questa non è ammissibile dato che $3 - 8 = -5$.

Prendiamo i casi misti invece: $(1, 5)$ (floor a sinistra e ceiling a destra) non è ammissibile, invece per il viceversa $(2, 4)$ ho l'ammissibilità ma non l'ottimalità. Infatti $Z(2, 4) = 4.56 \leq Z(5, 0) = 5$. Osserviamo la disuguaglianza!



Pertanto, appare che la soluzione al problema PL è "*superfluo*" al nostro problema IP. Tuttavia, vedremo che non sarà così, adoperando un altro approccio.

X

2. Approssimanti delle IP

IDEA. Dato un IP $z = \max \{c(x) : x \in X \subseteq \mathbb{Z}^d\}$, come possiamo trovare la soluzione ottimale $x^* \in \mathbb{Z}^d$? Un approccio comune è quello di trovare una *successione di approssimanti* finché non siano "*abbastanza*" vicini alla vera soluzione dell'IP.

#Definizione

Definizione (approssimante dal basso e dall'alto di una soluzione).

Sia z un valore ottimale di una IP.

Allora un'*approssimante dall'alto* di z è un valore $\bar{z}$ tale che $\bar{z} \geq z$. Analogamente un'*approssimante dal basso* $\underline{z}$ è $\underline{z} \leq z$.

Ossia abbiamo

$$\underline{z} \leq z \leq \bar{z}$$

In particolare vorremmo identificare delle successioni monotone (in particolare $(\underline{z}_n)_n$ è crescente e $(\bar{z}_n)_n$ è decrescente). Definendo tol o ε un "*valore di tolleranza*", possiamo concludere la ricerca con

$$\bar{z}_n - \underline{z}_k \leq \text{tol} (o \varepsilon)$$

Andiamo a cercare gli *approssimanti* degli IP.

X

3. Approssimanti dal Basso

#Proposizione

Proposizione (approssimanti dal basso di una IP).

Sia IP un problema di programmazione intera (WLOG di massimizzazione) con regione ammissibile X . Allora un qualsiasi elemento $\hat{x} \in X$ è un approssimante dal basso della soluzione:

$$c^T \hat{x} \leq c^T x^*$$

Questo è dovuto banalmente alla definizione di *valore ottimo* nel contesto di un problema di massimizzazione.

X

4. Approssimanti dall'Alto

Trovare gli approssimanti dall'alto di un problema IP (di massimizzazione, WLOG) non è invece così *banale*. L'idea comune è quello di rimpiazzare il problema "*difficile*" IP con un problema più "*semplice*" PL, dove il valore ottimo è almeno grande z . Questo concetto è noto come "*rilassamento*", che è fatto allargando l'insieme delle soluzioni ammissibili o rimpiazzando la funzione obiettivo max con una funzione che ha valori uguali o più grandi ovunque.

#Definizione

Definizione (rilassamento).

Sia *IP* un problema di programmazione intera del tipo $z = \max\{c^T x : x \in X \subseteq \mathbb{Z}^d\}$. Un suo *rilassamento RP* è un problema del tipo $z = \max\{f(x) : x \in T \subseteq \mathbb{R}^d\}$ tale che $X \subseteq T$ (ovvero T "*domina*" X) e $\forall x \in X, f(x) \geq c^T x$.

Denotiamo il valore ottimale del *RP* con z^R .

#Proposizione

Proposizione (il rilassamento costituisce un upper bound).

Si ha che, dato x^* soluzione ottima dell'IP, che dato il suo RP

$$c^T x^* \leq f(x^*) \leq z^R$$

#Dimostrazione

DIMOSTRAZIONE della [Proposizione 4](#)

La dimostrazione è omessa, in quanto banale. ■

Adesso diamo la *definizione canonica* del rilassamento *continuo lineare*.

#Lemma

Lemma (lemma preliminare).

Sia $P \subseteq \mathbb{R}^d$. Allora $P \cap \mathbb{Z}^d \subseteq P$.

#Definizione

Definizione (rilassamento continuo lineare).

Dato un problema IP $\max\{c^T x : x \in X = P \cap \mathbb{Z}^d\}$ definiamo il suo *rilassamento continuo lineare* RPL come

$$\max\{c^T x : x \in P\}$$

Chiaramente questo è in rilassamento con $T = P$ e $X = P \cap \mathbb{Z}^d$ e $f(x) = c^T x$.

Facciamo la seguente osservazione generale sui approssimanti:

#Osservazione

Osservazione (gli approssimanti possono essere arrotondati in certe condizioni).

In certe condizioni, dati degli approssimanti $\underline{z}, \bar{z}$ di z , possiamo *arrotondarli* per dare una stima migliore. In particolare si tratta di "*sostituire*" $\underline{z} \leftarrow \lceil \underline{z} \rceil$ e $\bar{z} \leftarrow \lfloor \bar{z} \rfloor$.

Si può fare se i *coefficienti* $c \in \mathbb{R}^d$ sono *esclusivamente interi*, ovvero sono $c \in \mathbb{Z}^d$. Questo è dovuto al fatto che il valore target dev'essere *intero* (in quanto lo scaling per interi o somme tra interi è un intero).

Adesso notiamo che possiamo definire la nozione di "*rilassamenti*" migliori, i.e. che hanno l'insieme delle soluzioni ammissibili più stretti, da cui ho che gli approssimanti dall'alto sono migliori.

#Proposizione

Proposizione.

Siano $P_1, P_2 \subseteq \mathbb{Z}^d$ delle *regioni ammissibili* per il problema di ottimizzazione IP1, IP2 con i stessi pesi $c \in \mathbb{R}^d$.

Allora definendo $\bar{z}_i$ le *soluzioni* ai problemi rilassati continui RPL1, RPL2, abbiamo che

$$P_1 \subset P_2 \implies \bar{z}_1 \leq \bar{z}_2$$

La *migliore* formulazione di RPL avviene quando la regione X è proprio l'*inviluppo convesso* di T .

X

5. Relazioni di Esiti

#Proposizione

Proposizione (condizione sufficiente per l'innammissibilità).

Sia IP un problema di programmazione intera. Se il suo rilassamento continuo lineare RPL è *innammissibile* allora lo è pure IP.

#Dimostrazione

DIMOSTRAZIONE della [Proposizione 9](#)

Chiaramente sia $X = \emptyset$ la regione di RPL, siccome $X \subseteq T$ allora $T = \emptyset$. ■

#Proposizione

Proposizione (soluzioni ottime uguali).

Sia IP un problema di programmazione intera e RPL il suo rilassamento continuo lineare.

Se la soluzione dell'RPL x^* è *intero*, i.e. $x^* \in \mathbb{Z}^d$, allora questa è anche la soluzione per IP.

Matrice Totalmente Unimodulare

X

Definizione di matrice totalmente unimodulare. Condizioni equivalenti per matrici TU; condizioni sufficienti. Esempi, controesempi.

v

0. Voci correlate

- Problemi IP Ben Posti

1. Definizione di Matrice TU

#Definizione

Definizione (matrice totalmente unimodulare).

Una matrice $A \in \mathbb{R}^{n \times m}$ si dice *unimodulare* sse $\forall a \subseteq A$ vale che $\det a \in \{-1, 0, 1\}$.

Notiamo che A TU implica che ogni elemento $A[i, j]$ appartiene all'insieme $\{-1, 0, 1\}$; tuttavia non vale l'inversa, infatti

$$A = \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix}$$

ha determinante $\det A = 2$.

#Proposizione

Proposizione (condizione necessaria per le matrici TU).

Se una matrice A è *TU* allora $\forall i, j, A[i, j] \in \{-1, 0, 1\}$.

X

2. Condizioni Equivalenti

#Proposizione

Proposizione (condizioni equivalenti per matrici TU).

Una matrice A è *TU* se e solo se:

- La trasposta A^T è *TU*
- La matrice completa $A|1$ è *TU*

X

3. Condizioni Sufficienti

Proposizione (condizioni sufficienti per matrici TU).

Una matrice A è **TU** se vale che:

- i) $\forall i, j, A[i, j] \in \{-1, 0, 1\}$
- ii) Ogni colonna contiene **al più** due coefficienti non zero, ossia $\forall j, \sum_i A[i, j] \leq 2$.
- iii) Esiste una partizione (M_1, M_2) delle M righe tali che ogni colonna j soddisfacente ii) soddisfa

$$\sum_{i \in M_1} A[i, j] = \sum_{i \in M_2} A[i, j]$$

La condizione iii) ci dice che in ogni **colonna** tutti i valori non-zero devono essere tali che:

- Se segni discordi, allora appartengono alla stessa classe
- Se segni concordi, allora appartengono a classi diverse

Facciamo un esempio: sia $A \in \mathbb{R}^{5 \times 7}$, definita come

$$A = \begin{pmatrix} -1 & 0 & 1 & 0 & -1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 0 & 1 \\ -1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & -1 & 0 \end{pmatrix}$$

Denotando $R_i = A[i, :]$ la riga i -esima, vogliamo individuare una partizione per $R = \{R_1, \dots, R_5\}$ che soddisfi la condizione del teorema. Per farlo, procediamo colonna per colonna:

- **Colonna** $A[:, 1]$: $A[1 : 1] = A[3 : 1]$, pertanto R_1, R_3 appartengono a classi diverse
- **Colonna** $A[:, 2]$: Analogamente R_2, R_3 appartengono a classi diverse. Per dicotomia R_1, R_2 appartengono alla stessa classe
- **Colonna**: Operazioni analoghe

Continuando otteniamo $M_1 = \{R_1, R_2\}, M_2 = \{R_3, R_4, R_5\} : M = \bigsqcup_{i=1,2} M_i$.

Inoltre osserviamo che è possibile che ci sia $M_1 = \emptyset$ e $M_2 = M$.

Esercizio (esercizio).

Verificare che la seguente matrice è TU

$$\begin{pmatrix} 1 & -1 & -1 & 0 \\ -1 & 0 & 0 & 1 \\ 0 & 1 & 0 & -1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

Cosa noti con le partizioni M_1, M_2 ?

Se vediamo un caso che non soddisfa la iii), è comunque possibile che la matrice sia TU.

Problemi IP Ben Posti

X

Definizione di problema IP ben posto, caratterizzazione dei problemi IP ben posti.

X

0. Voci correlate

- [Approssimanti di Soluzioni IP](#)
- [Matrice Totalmente Unimodulare](#)

1. Problemi IP Ben Posti

Q. Sia (IP) un problema di *programmazione intera* e (RPL) il suo rilassamento continuo. Quando si ha che (IP) ha le stesse soluzioni di (RPL)?

#Definizione

Definizione (problema ben posto).

Un problema (IP) è ben posto se vale che la sua soluzione z^* sono le stesse del suo rilassamento continuo (RPL).

Caratterizziamo i problemi *IP* ben posti.

#Teorema

Teorema (caratterizzazione dei problemi ben posti).

Un problema (IP) posta come (IP) : $\max\{c^T x : Ax \leq b\}$ è *ben posta* se e solo se A è *totalmente unimodulare*.

Problemi IP Ben Posti Notevoli

X

Esempi di problemi IP ben posti. Problema del flusso di costo minimo (MCNF). Proposizione: MCNF ha la matrice dei coefficienti TU. Problemi derivanti da MCNF: il problema del cammino più breve (TSP), il problema del flusso massimo (MFP), il problema del trasporto (TP), il problema dell'assegnazione (AP).

X

0. Voci correlate

- [Matrice Totalmente Unimodulare](#)
- [Problemi IP Ben Posti](#)

1. Problema del Flusso di Costo Minimo (MCNF)

Sia $G = (V, E)$ un *grafo direzionato*. Denotiamo, per ogni nodo i il suo flusso associato "uscente" ed "entrante":

- $V_i^+ = \{k : (i, k) \in E\}$
- $V_i^- = \{k : (k, i) \in E\}$

#Definizione

Definizione (grafo di un flusso).

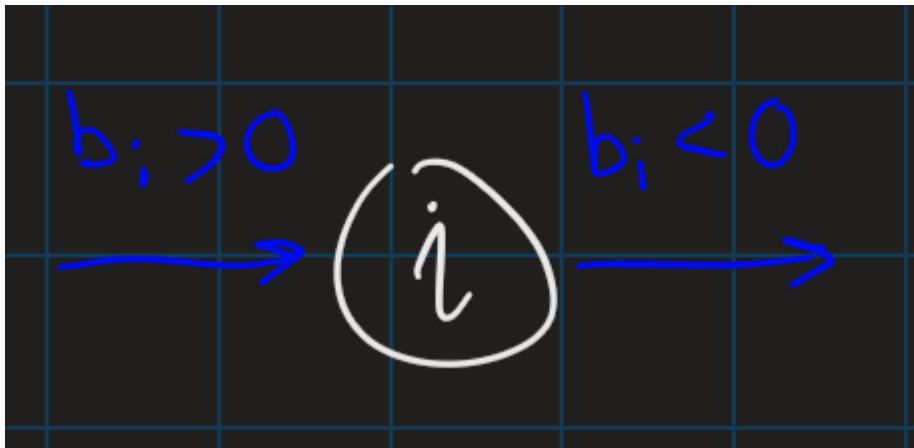
Un grafo direzionato $G = (V, E)$ rappresenta un *rete di flusso* se:

- Esistono due funzioni $h, c : E \rightarrow \mathbb{R}$, detti *capacità* e *costo* associato ad ogni arco. Denotiamo un suo valore qualsiasi con $h_{i,j}$
- Esiste una funzione $b : V \rightarrow \mathbb{R}$, detto *domanda* di ogni nodo. Denotiamolo con b_i

In tal caso (G, h, c, b) è una *rete di flusso*.

Notiamo che per $b_i \neq 0$, ho:

- Se $b_i > 0$, allora in i *entra* quella quantità di flusso da "fuori" dal grafico
- Se $b_i < 0$, allora da i *esce* quella quantità di flusso in "fuori" del grafico



#Definizione

Definizione (problema del flusso di costo minimo).

Sia (G, h, c, b) una *rete di flusso*. Sia x_{ij} la *quantità di flusso* imposta nell'arco $(i, j) \in E$. Il *problema di ottimizzazione* posto come

$$\begin{aligned} \min \quad & \sum_{(i,j) \in E} c_{ij} x_{ij} \\ \forall i \in V, \quad & \sum_{k \in V_i^+} x_{ik} - \sum_{k \in V_i^-} x_{ki} = b_i \quad (1) \\ \forall (i, j) \in E, \quad & 0 \leq x_{ij} \leq h_{ij} \end{aligned}$$

Si dice il *problema del flusso di costo minimo*. La (1) è nota come il *principio di conservazione* (o *legge di Kirchhoff*), e se è vera allora la domanda è soddisfatta. Inoltre $x_{ij} \leq h_{ij}$ impone le condizioni di capacità

Abbiamo che i problemi di questo tipo sono *ben posti*.

#Proposizione

Proposizione (MCNF è ben posto).

Ogni problema ammissibile *del flusso di costo minimo* è *ben posto*.

#Dimostrazione

DIMOSTRAZIONE del [Proposizione 3](#)

Notiamo che la maggior parte dei coefficienti di A vengono dal *vincolo di conservazione*. Infatti, gli altri vincoli vengono dai vincoli di capacità tutti a coefficiente unitario. Pertanto $A = (C, \mathbb{1})^T$. Pertanto per dimostrare la tesi basta provare che A è *TU*, ossia sse C è *TU* ([Proposizione 3](#)). La dimostrazione di C *TU* è omessa, tuttavia osserviamo che ogni coefficiente è in $\{-1, 0, 1\}$.

Adesso iniziamo a vedere un po' di *problemi* di programmazione intera che siano *riducibili* al *MCNF*.

2. Problema del Cammino Più Breve (TSP)

#Definizione

Definizione (problema del cammino più breve).

Sia $G = (V, E)$ un *grafo direzionato* e siano $s, t \in V : s \neq t$. Definendo dei "*pesi*" ad ogni arco $w : E \rightarrow \mathbb{R}^+$, il problema di *trovare* un cammino da s a t che minimizzi il peso totale si dice il *problema del cammino più breve*.

#Corollario

Corollario (TSP è ben posto).

Ogni problema *TSP ammissibile* è *ben posta*.

#Dimostrazione

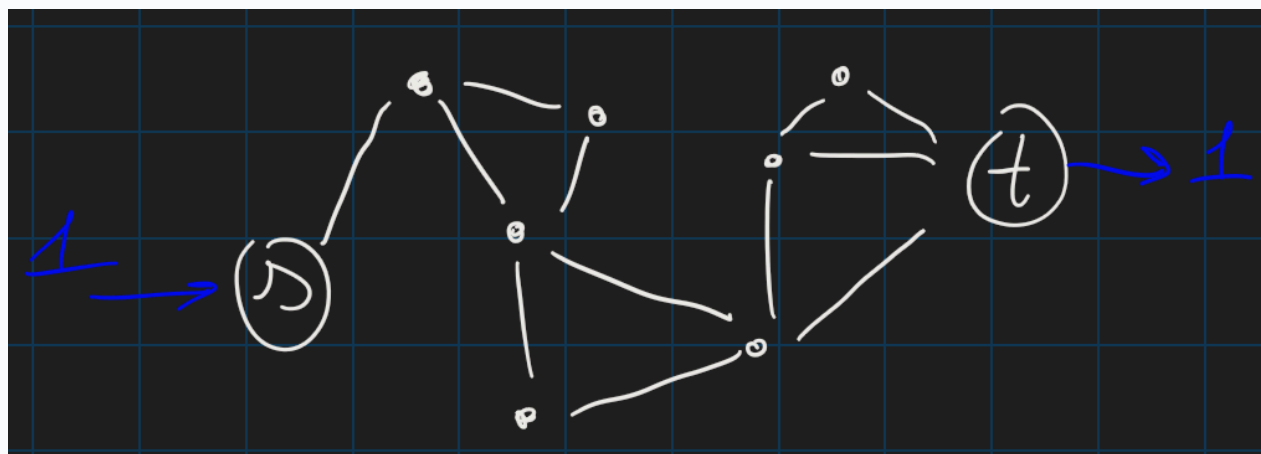
DIMOSTRAZIONE del Corollario 5

Si tratta di ricondurci al [Proposizione 3](#) (ovvero alla *MCNF*). Ciò vuol dire che ovviamente $w_{ij} = c_{ij}$ e che dobbiamo trovare opportunamente la funzione h_{ij} e b_i

Notiamo che nel nostro contesto è opportuno prendere x_{ij} come un *valore binario*, ossia decidere "*se attraversare l'arco (i, j) o meno*" (i cicli non avrebbero senso). Pertanto $\forall (i, j) \in E, h_{ij} = 1$ (è l'estremo superiore opportuno).

Come possiamo decidere b invece? Adesso intervengono i nodi "*target*", ossia s di partenza e t finale. Notiamo che definendo $b_s = 1$ e $b_t = -1$, abbiamo che c'è una quantità di flusso unitaria che deve entrare da s e uscire in t ; ovvero, deve trovare il cammino. Per il restante, poniamo $\forall i \notin \{s, t\}, b_i = 0$.

Notando che abbiamo una *MCNF*, concludiamo. ■



3. Problema del Flusso Massimo

#Definizione

Definizione (problema del flusso massimo).

Sia $G = (V, E)$ un *grafo direzionato* su cui definiamo le capacità h_{ij} . Siano $s \neq t$ dei nodi.

Un problema del *massimo flusso* è la *ricerca delle quantità di flusso* ponibili da s a t , purché rispettando il principio di conservazione, tale che massimizzi la quantità totale portata.

#Corollario

Corollario (MFP è ben posta).

Tutti i problemi del flusso massimo sono ben posti.

#Dimostrazione

DIMOSTRAZIONE del Corollario 7

In questo caso ci riconduciamo al *problema del cammino più breve*, che a sua volta si riconduce al problema del *flusso di costo minimo*. Definiamo le variabili decisionali x_{ij} : quantità di flusso posto sull'arco (i, j) e h_{ij} è banale in quanto è già posto nel problema; pertanto ci rimane da definire i vincoli e la funzione obiettivo.

Vincoli: Ovviamente dobbiamo far valere il principio di conservazione negli archi interni, ovvero

$$\forall i \in V \setminus \{s, t\}, \sum_{k \in V_i^+} x_{ik} = \sum_{k \in V_i^-} x_{ki}$$

Tuttavia notiamo che $x_{ij} \leq \min\{\sum_{i \in V_s^+} x_{si}, \sum_{i \in V_t^-} x_{it}\} := M$, che è l'upper bound delle soluzioni possibili.

Funzione Obiettivo: Basta massimizzare

$$\max \sum_{(i,j) \in E} x_{ij}$$

Come possiamo *ricondurci* al problema del massimo in un problema del minimo?

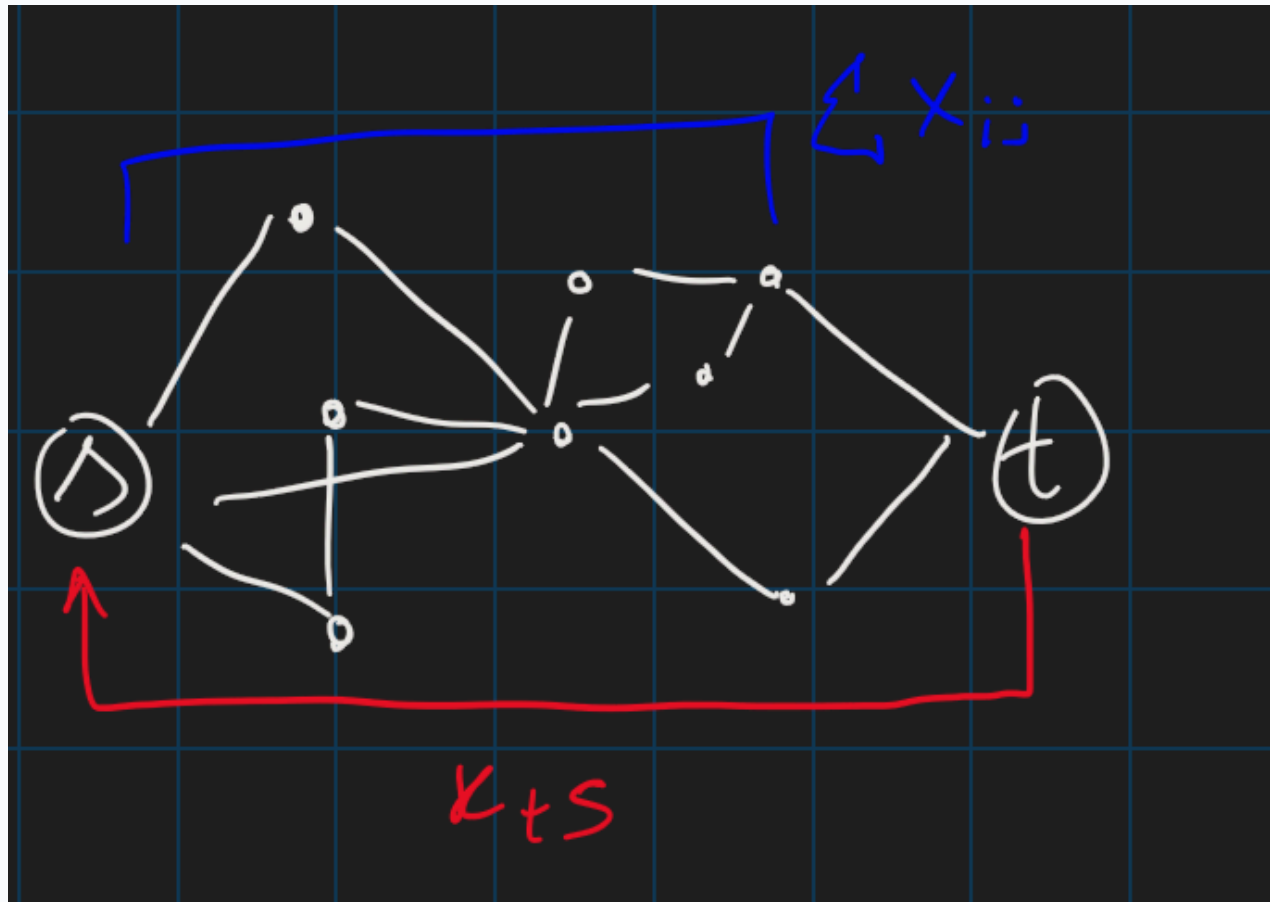
Definiamo un *"arco fittizio"* x_{ts} che va da t a s in una maniera diretta, con capacità massima M ($0 + \infty$), ossia $h_{ts} = +\infty$, e con costo -1 (in quanto è un arco *fittizio*); gli altri costi sono posti a 0 , in quanto sono attraversabili senza problemi. Allora il problema di massimizzazione si esprime come

$$\max \sum_{(i,j) \in E} w_{ij} x_{ij} = \max \sum_{(t,s) \neq (i,j) \in E} 0 \cdot x_{ij} - x_{ts} = \max -x_{ts}$$

Questo diventa il problema di minimizzazione

$$\max -x_{ts} = \min x_{ts}$$

concludendo. ■



X

4. Problema del Trasporto

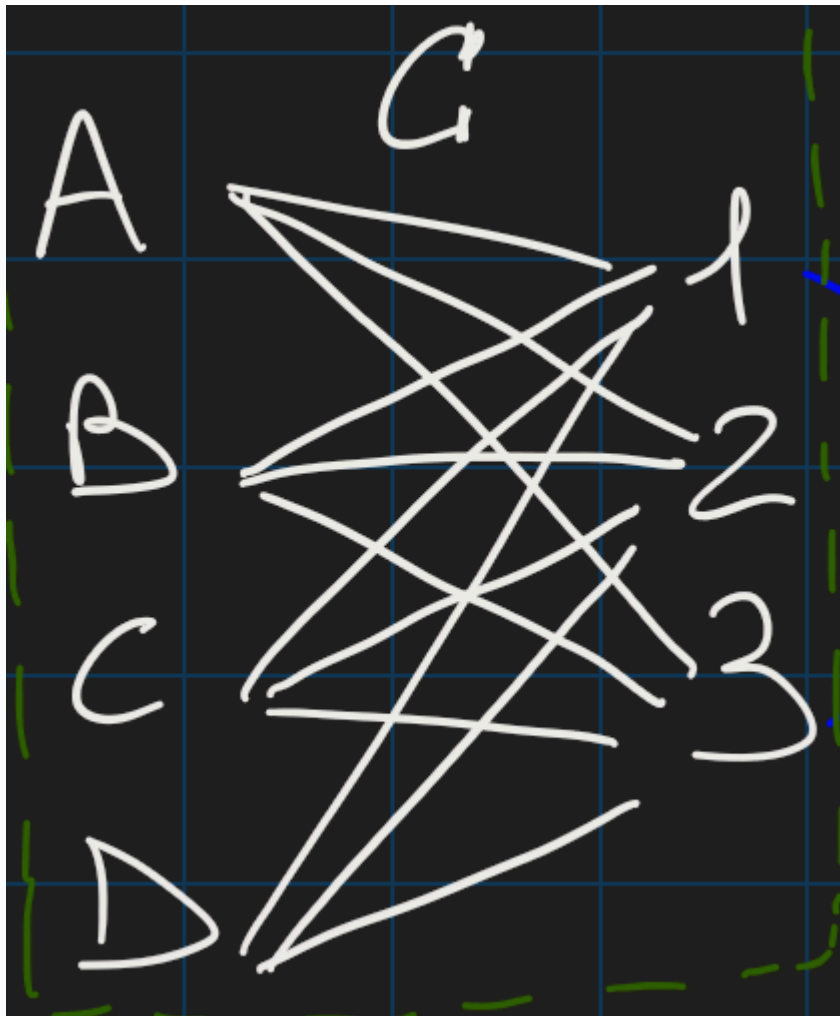
#Definizione

Definizione (problema del trasporto).

Sia $G = (V, E)$ un *grafo bipartito direzionato*, i.e. $V = V_1 \sqcup V_2$ tali che $E = \{(i, j) : i \in V_1, j \in V_2\}$ (ossia abbiamo "*due layer*" del grafo pienamente connessi).

Definiamo V_1 i nodi *sorgenti* e V_2 i nodi *pozzi*. Definiamo il "*costo di spedizione*" $c : E \rightarrow \mathbb{R}^+$ associato ad ogni arco, la *produzione* $a : V_1 \rightarrow \mathbb{Z}^+$ e la *domanda* $B : V_2 \rightarrow \mathbb{Z}^+$. Se l'arco (i, j) non esiste, allora si definisce c_{ij} un numero arbitrariamente alto.

Il problema di *trovare le spedizioni* che minimizzino i costi totali si dice il *problema di trasporto*.



#Corollario

Corollario (TP è ben posto).

Il *problema del trasporto* (purché ammissibile) è ben definito.

#Dimostrazione

DIMOSTRAZIONE del Corollario 9

L'idea è quello di ricondurci al problema del *flusso massimo* (Corollario 7).

Notiamo che ogni "*flusso outbound*" da una sorgente dev'essere *uguale* alla sua *produzione* e ogni *domanda* da un pozzo dev'essere sempre soddisfatta. Pertanto vale che

$$\sum_{i \in V_1} a_i = \sum_{j \in V_2} b_j$$

Definiamo $x_{ij} : V_1 \times V_2 \longrightarrow \mathbb{R}^+$ la variabile decisionale come "*la quantità di flusso*" portata in E . Da questo abbiamo la funzione obbiettivo

$$\min \sum_{(i,j) \in E} c_{ij} x_{ij}$$

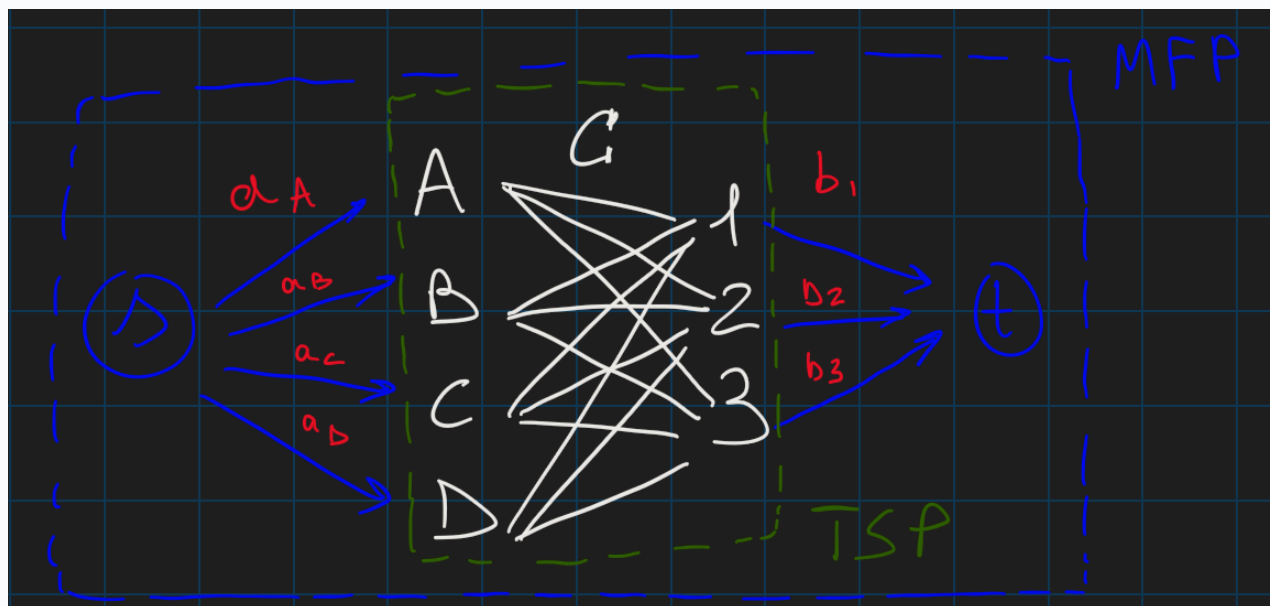
Poi *definiamo* due condizioni sul problema: la soddisfacibilità della *domanda* e della *produzione*. Ossia

$$\forall i \in V_1, \sum_{j \in V_2} x_{ij} \leq a_i$$

$$\forall j \in V_2, \sum_{i \in V_1} x_{ij} \geq b_j$$

Notiamo che le disuguaglianze possono essere sostituite con delle uguaglianze strette per l'equazione (1), ossia per la conservazione.

Come facciamo a ricondurci al problema *MFP*? L'idea è quello di *aggiungere* due nodi fittizi agli "*estremi*" del nostro grafo bipartito e di collegarli ai layer V_1, V_2 . Ossia definiamo $s \in V$ tale che $\forall i \in V_1, (s, i) \in E$ e analogamente $t \in V$ t.c. $(j, t) \in V_2$. Definiamo i costi *nulli* e le capacità $\forall i \in V_1, h_{si} = a_i$ e $\forall j \in V_2, h_{jt} = b_j$. In questo modo abbiamo una *MFP* che vada a comprendere i dati iniziali, e minimizzandola si minimizzano *solo* gli archi interni, in quanto quelli "*fittizi*" hanno costo zero. ■



Osserviamo che di solito i *dati* vengono forniti con *vettori* e *matrici*, i.e. dati p sorgenti e c pozzi, abbiamo:

- $C \in \mathbb{R}^{p \times c}$ la *matrice dei costi*, dove $C[i, j]$ indica il *costo di spedizione* dal sorgente i al pozzo j
- $a \in \mathbb{Z}_+^p, b \in \mathbb{Z}_+^c$ i vettori *capacità* e *domanda*. Osserviamo che per equazione (1) abbiamo $\|a\|_1 = \|b\|_2$

Per $p = c$ e $a = b = J$ (il vettore con tutti uno) abbiamo il *problema dell'assegnazione*, in cui modifichiamo il vincolo della *domanda*:

$$\forall j \in V_2, \sum_{i \in V_1} x_{ij} \leq 1$$

Questo è dovuto alla motivazione euristica, per cui "*ogni risorsa*" può svolgere al più "*un lavoro*".

Algoritmo Branch & Bound

Algoritmo Branch & Bound per risolvere problemi IP qualunque. Idea divide et impera di B&B. Proposizioni preliminari: suddivisione di regioni ammissibili, maggiorazione delle soluzioni nelle regioni ammissibili. Tre criteri di potatura: ottimalità, maggiorazione, innammissibilità. Algoritmo.

0. Voci correlate

- [Approssimanti di Soluzioni IP](#)

1. Osservazioni Preliminari

Q. Dato un *problema di programmazione intera* (IP), abbiamo due approcci per risolverlo:

- Se è un problema ben posto, allora risolviamo il suo rilassamento lineare RPL
- Altrimenti usiamo un algoritmo *ad hoc*, per problemi di questo tipo

L'algoritmo *Breach & Bound* va a coprire la seconda casistica, con un *algoritmo esatto*: con risorse e tempo infinito, la risoluzione del problema è garantita.

IDEA. Usiamo un approccio "*divide et impera*" per esplorare l'insieme delle soluzioni ammissibili S . Ossia, creiamo ricorsivamente delle suddivisioni di S finché non otteniamo dei problemi "*facilmente risolvibili*". Enunciamo dunque la seguente proposizione:

#Proposizione

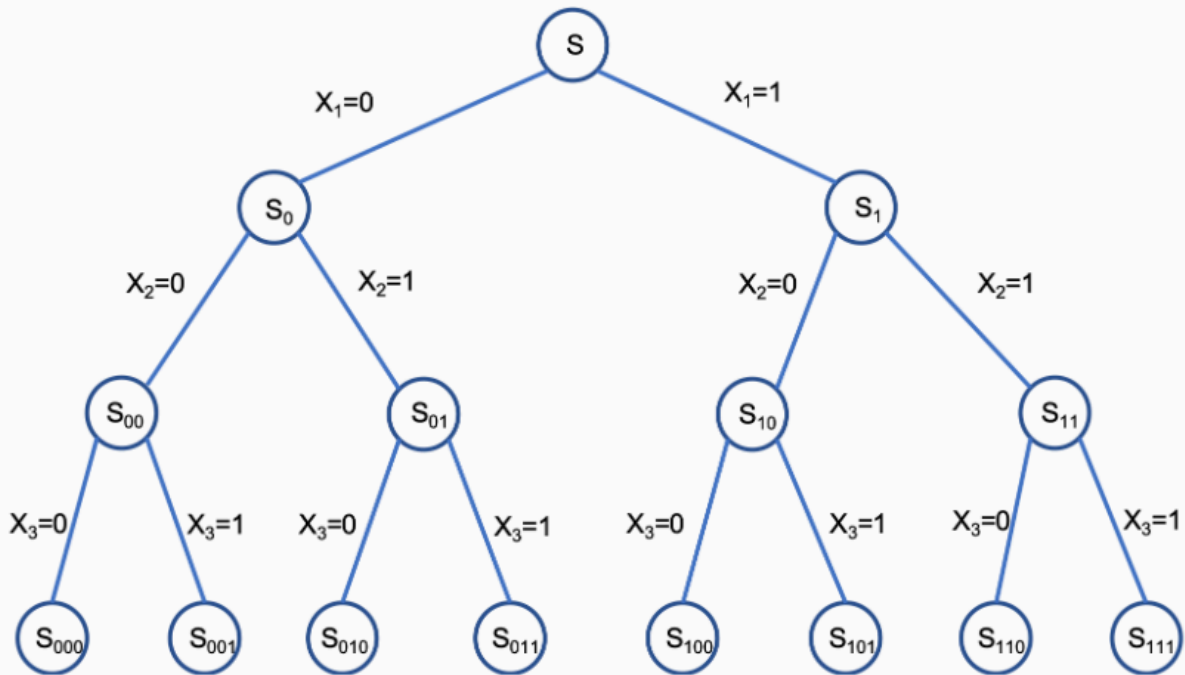
Proposizione (suddivisione di problemi di programmazione).

Sia $S = \cup_n S_n$ una composizione di S in sottoinsiemi più piccoli. Definiamo $z_k^* := \max\{c(x) : x \in S_k\}$, ovvero il valore ottimale (WLOG) del problema k -esimo.

Allora

$$z^* = \max_k z_k^*$$

Un modo classico per rappresentare tale approccio è quello di usare *alberi di enumerazione*, dove "*splittiamo*" S fissando una condizione su una variabile decisionale. Tuttavia, per disegnare *interamente* degli alberi di enumerazioni, abbiamo bisogno di un numero elevato di nodi.



Quindi, enuncieremo preliminarmente il seguente risultato, per definire bene un "*criterio intelligente*" di arresto.

#Proposizione

Proposizione (maggiorazione delle suddivisioni).

Sia $S = \cup_n S_n$ una suddivisione di S . Sia $z_k^* := \max\{c(x) : x \in S_k\}$.

Allora si ha che:

Il *massimo degli approssimanti in alto* è un limite superiore di z^* :

$$\bar{z} := \max_k \bar{z}_k \geq z^*$$

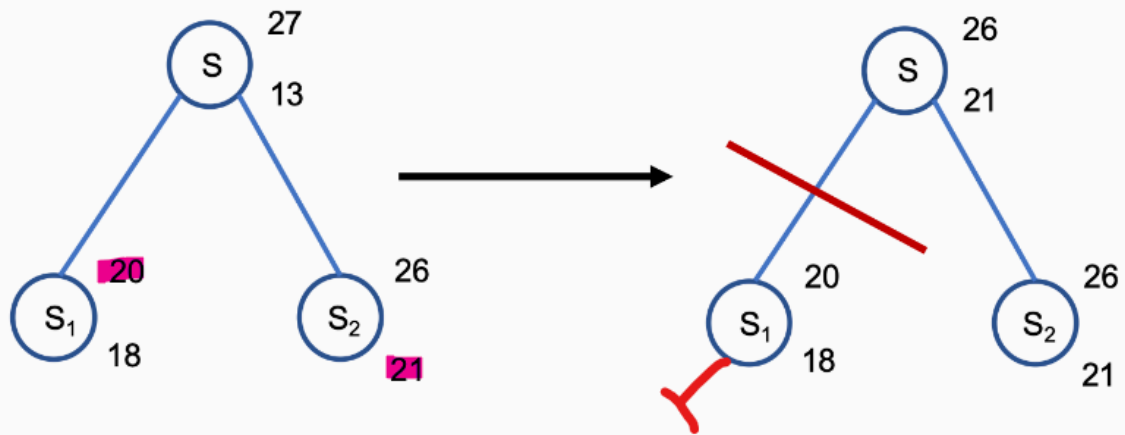
Inoltre anche il *massimo degli approssimanti in basso* è un limite inferiore di z^*

$$\underline{z} = \max_k \underline{z}_k \leq z^*$$

Il primo punto è intuitivo ed è facilmente ricavabile facendo dei rilassamenti continui. Tuttavia il secondo punto è meno intuitivo; si sceglie di prendere il *massimo* in quanto un'approssimante dal basso è rappresentata da una soluzione qualunque, quindi vado a scegliere la "*migliore*" approssimazione dal basso.

Pertanto, data S_1, S_2 suddivisione di S , se abbiamo che $\bar{z}_1 \leq \underline{z}_2$, allora S_1 diventa "*inutile*" da esplorare e dunque si può decidere di non dividere ulteriormente S_1 . Questa è nota come il *criterio d'arresto per limitazione*.

MAX



X

2. Algoritmo Branch & Bound

Per definire un *algoritmo ben posto*, ci chiediamo le seguenti domande:

- Come troviamo degli *upper bound*? Usiamo il rilassamento continuo o la dualità dei problemi?
- Conviene più scegliere di trovare *approssimanti deboli, ma veloci da calcolare* o *approssimanti forti, ma lenti da calcolare*?
- Come possiamo creare la suddivisione $S = \cup_n S_n$?
- Facciamo *solo* due split (creando pertanto una B-tree), o di più? E *come* facciamo gli split?
- In che ordine dobbiamo esaminare i sottoproblemi S_i ? Con quale criterio dobbiamo scegliere?

Un algoritmo *qualunque* Branch & Bound tenta di fornire una risposta a queste domande, scegliendo delle *"euristiche"* precise. Tuttavia, possiamo delineare un *algoritmo generale* per *Branch & Bound*.

ALGORITMO. (*Branch & Bound, quasi-generico*)

Sia (IP) : $\{\max c^T x : x \in S\}$ il problema da risolvere. Allora, finché non tutti i sottoproblemi siano marcati *"arrestati"*, eseguiamo la seguente iterazione. Definiamo $S_0 = S$

- Dividere S_i in $(S_{i,1}, \dots, S_{i,k})$ sottoproblemi
- Per ogni sottoproblema (denotiamo con $S_{i,j}$):
 - Se ha regione ammissibile vuota, marcare $S_{i,j}$ chiuso (*condizione di arresto per inammissibilità*)
 - Altrimenti calcolare il suo *limite inferiore e superiore* $\bar{z}_{i,j}$ e $\underline{z}_{i,j}$
 - Per calcolare $\bar{z}_{i,j}$, di solito si prende il valore ottimale del rilassamento continuo lineare

- Se non è possibile calcolare $\underline{z}_{i,j}$ per mancanza di conoscenza di punti ammissibili, porlo per convenzione a $\underline{z}_{i,j} = -\infty$
 - Se $\bar{z}_{i,j} = \underline{z}_{i,j}$, marcare $S_{i,j}$ chiuso (*condizione di arresto per ottimalità*)
- Rivedere ogni sottoproblema, in particolare per ogni $\bar{z}_{i,j} \leq \bar{z}_{i,k}$, marcare $S_{i,j}$ chiuso (*condizione di arresto per "bounding"*)
- Marcare S_i chiuso, andare al prossimo nodo da esplorare

Infine, iterare su tutti i nodi chiusi per ottimalità, e prendere quella che contiene il miglior bound. ■

Notiamo che questo algoritmo è *quasi generico* in quanto implicitamente va a implementare una ricerca *BFS*, di solito il caso più conveniente in quanto facciamo un branching binario. Per rendere il caso più generico si dovrebbe usare altre strutture dati come *pile*.

Vediamo un *paio di esempi di euristiche*, usati comunemente.

- *Branching*: Di solito, facciamo il *branching* su una singola variabile decisionale x_i . In particolare supponiamo di avere calcolato la *soluzione* del rilassamento continuo; se *una delle componenti* è *non-intera* (ossia frazionale), allora questa diventa il *"baricentro"* del branching.
- *Branching (binario)*: Se invece abbiamo variabili binarie, allora il branching diventa banale, basta dividere per *"sì"* o *"no"*.
- *Upper Bound (binario)*: Per il problema noto *Knapsack-Problem (KP)*, usiamo l'upper bound di Dantzig, definito sotto.
- *Visita dei nodi*: In quale ordine vediamo i problemi? Eseguiamo una *DFS* o una *BFS*?

#Definizione

Definizione (limite superiore di Dantzig).

Sia $N > 0$ fissato. Siano $(p_n)_{n \leq N}$, $(w_n)_{n \leq N}$ delle sequenze che rappresentino rispettivamente il *"gradimento"* e il *"peso"* dell'oggetto n -esimo. Sia $W > 0$ il *"peso massimo"*

Sia $\rho_n := \frac{p_n}{w_n}$ la *"densità di gradimento"*. Adesso prendiamo un riordinamento di $(\rho_n)_n$ tale che questa diventi *crescente*. Ossia gli indici sono tali che

$$\rho_1 \geq \rho_2 \geq \dots \geq \rho_N$$

Sia $r > 0$ il numero intero tale che

$$\sum_{j=1}^{r-1} w_j \leq W \wedge \sum_{j=1}^r w_j > W$$

Definendo $x_j = 1$ per ogni $j < r$ e $x_j = 0$ per ogni $j > r$, e invece

$$x_r = \frac{W - \sum_{j=1}^{r-1} w_j}{w_r}$$

Infine definendo il "*peso eccesso*" data da

$$\bar{W} = W - \sum_{j < r} w_j$$

Si definisce l'*upper bound* di Dantzig come la quantità data da

$$\text{UB} := \left\lfloor \sum_{j < r} p_j + \rho_r \bar{W} \right\rfloor$$

Si dimostra che per un problema KP, si ha

$$\text{UB} \geq z^*$$

Algoritmi Greedy

X

Algoritmi greedy. Definizione, motivazioni. Esempi di algoritmi greedy: Problema 0/1 Knapsack. Osservazione: le soluzioni greedy non garantiscono nessun tipo di maggiorazione sull'errore. Altri esempi: problema del cammino più breve (algoritmo a più frammenti), algoritmo NN. Problema della ricerca del minimo albero coprente. Algoritmo greedy per la MST.

X

0. Voci correlate

- [Algoritmo Branch & Bound](#)
- [Approssimanti di Soluzioni IP](#)

1. Motivazione e Definizione di Algoritmi Greedi

Q. L'algoritmo *Branch & Bound* richiede, ad un sottoproblema, di trovare un suo *upper* e *lower bound* (se, assumendo WLOG che stiamo avendo a che fare con un problema di massimo, allora sono le soluzioni "*migliori*" e "*peggiori*"). Si trova un *upper bound* facilmente eseguendo un *rilassamento lineare* o usando altri algoritmi specifici. Come facciamo invece a trovare dei *lower bound*?

Gli algoritmi greedy ci aiuteranno a trovare delle "*buone soluzioni*", date in una maniera sistematica.

#Definizione

Un *algoritmo greedy* è una sequenza di scelte che siano *localmente ottimali*.

In altre parole, un *algoritmo greedy* vede il suo stato e impone come variabile come la scelta che ritorna la *miglior ricompensa immediata*.

Vedremo degli esempi.

X

2. Problema Knapsack

Sia dato un *problema Knapsack* caratterizzato da $n > 0$ oggetti, pesi $w_1, \dots, w_n$, gradimenti $p_1, \dots, p_n$ e peso massimo $W > 0$. Supponiamo, senza perdere di generalità, che gli oggetti siano ordinati rispetto alla loro "*densità*" $\rho_i := p_i/w_i$ nel senso decrescente (così favoriamo prima gli oggetti che hanno "*peso piccolo*" e "*valore maggiore*" contemporaneamente).

Allora Dantzig ci ha fornito una formula per calcolare l'*upper bound* senza dover ricorrere alla programmazione lineare.

Come troviamo un *lower bound*?

IDEA. Il nostro algoritmo semplicemente va ad inserire quanti oggetti possibili, in ordine.

ESEMPIO. Dato il problema

$$\begin{array}{ll}\max & 12x_1 + 8x_2 + 17x_3 + 11x_4 + 6x_5 + 2x_6 + 2x_7 \\ \text{s. t.} & 4x_1 + 3x_2 + 7x_3 + 5x_4 + 3x_5 + 2x_6 + 3x_7 \leq 9 \\ & x_j \in \{0, 1\} \quad \text{for } j = 1, \dots, 7\end{array}$$

Allora il primo, terzo e il penultimo oggetto possono essere inseriti ($4 + 3 + 2 = 9$).

Pertanto un lower bound della soluzione sarà

$$(1, 1, 0, 0, 0, 1, 0)$$

OSSERVAZIONE. Notiamo che non è *garantito in nessun modo* che la soluzione approssimata sia "*vicina*" alla soluzione effettiva. Consideriamo la seguente parametrizzazione del problema KP:

$$\begin{array}{ll}\max & 2x_1 + Mx_2 \\ & x_1 + Mx_2 \leq M \\ & x_1, x_2 \in \{0, 1\}\end{array}$$

Questo è posto bene, infatti $\rho_1 = 2$ e $\rho_2 = 1$. Ovviamente, $M > 0$. Per $M > 2$ notiamo subito che la soluzione ottimale è $(0, 1)$. Tuttavia, con l'algoritmo greedy si potrebbe ottenere un *lower bound* potenzialmente basso, a seconda del valore M .

Denotando $z_g = 2$ la *soluzione dell'algoritmo greedy* e $z_* = M$ la soluzione esatta, si ottiene che

$$\lim_{M \rightarrow +\infty} \frac{z_g}{z_*} = 0$$

Analogamente, potrebbe tendere ad un qualsiasi valore arbitrario ≤ 1 .

X

3. Problema del Commesso Viaggiatore

Supponiamo di avere un problema del *commesso viaggiatore simmetrico*, i.e. considero un *grafo non direzionato* $G = (V, E)$. Come facciamo a trovare una *soluzione ammissibile*, ovvero un *ciclo Hamiltoniano*? Notiamo che in questo modo abbiamo un *upper bound*, siccome stiamo trattando un *problema di minimizzazione*!

Si tratta di trovare un *cammino Hamiltoniano*. Allora ho le seguenti due strategie:

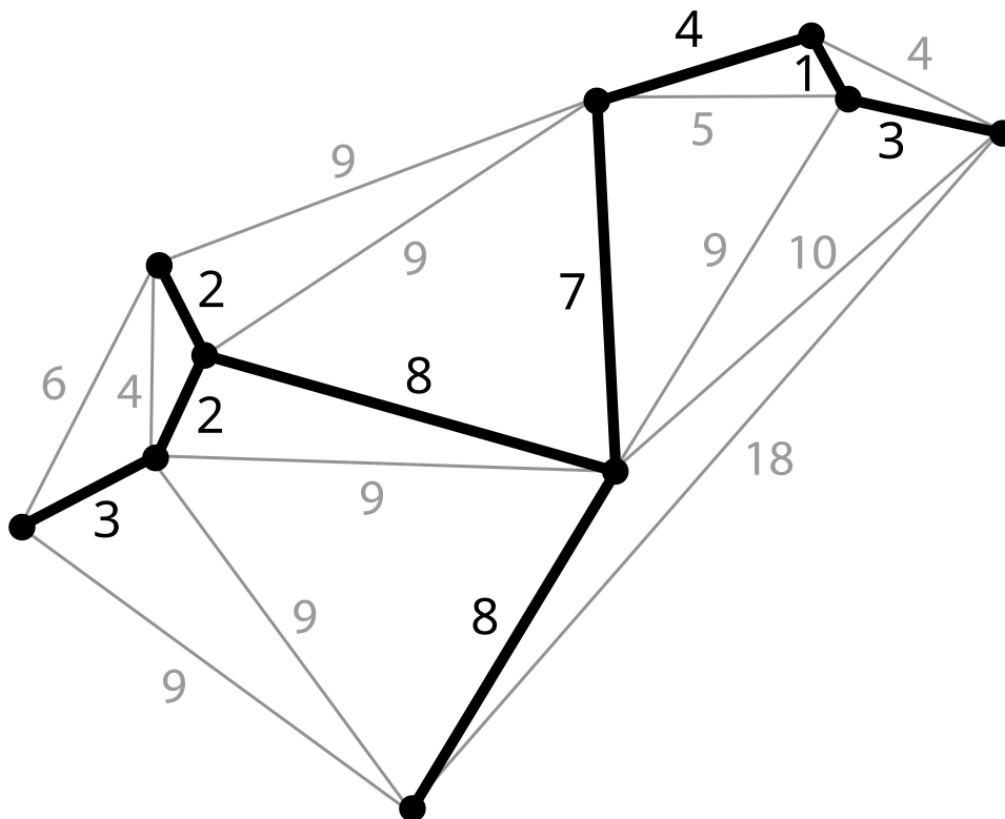
- Prendo archi più corti finché non inizio a creare sottocicli (*Multi-Fragment*)
- Cominciando da un "*nodo iniziale*" scelto, trovo percorsi meno costosi per i suoi nodi adiacenti evitando sottocicli.

Come sempre, non c'è nessuna forma di garanzia di avere una *relazione* tra la soluzione greedy e la vera soluzione ottimale.

X

4. Problema del Minimo Albero Coprente

Dato un grafo $G = (V, E)$ non diretto, definiamo un sottoinsieme di archi $E_T \subset E$ tale che $T = (V, E_T)$ è un *albero* (tocca tutti i nodi). Tale albero T si dice *albero coprente* di G



PROBLEMA. Dato un grafo non diretto $G = (V, E)$, trovare il suo *albero coprente* che vada a *minimizzare il costo totale dei coefficienti* $\sum_{(i,j) \in E_T} c_{i,j}$.

APPLICAZIONI.

- Progettazioni delle reti di comunicazione (*reti di calcolatori*)

FORMULAZIONE. Formuliamo il *problema del minimo albero coprente* (MST). Definiamo x_e dove $e \in E$, che è 1 se la nostra soluzione la include come arco o meno. Siccome un albero coprente ha *sempre* $|V| - 1$ archi (per evitare cicli!), abbiamo il primo vincolo

$$\sum_{e \in E} x_e = |V| - 1$$

Tra l'altro, gli archi scelti non devono generare *sottocicli*. Allora per ogni sottinsieme dei nodi $\emptyset \neq S \subset V$ definiamo gli "*archi su S*":

$$E_S = \{(i, j) \in E : i, j \in S\}$$

Allora, come prima dobbiamo avere che la somma degli archi scelti in E_S sia minore o uguale della sua dimensione. Ovvero

$$\forall S, \sum_{e \in E_S} x_e \leq |S| - 1$$

Notiamo che con il vincolo (1) otteniamo un numero *esponenziale di vincoli* (in quanto i sottinsiemi di V sono composti da 2^V). Pertanto ottenere un *lower bound* mediante un *algoritmo greedy che lo risolva "velocemente"* (in specie di tempo *polinomiale*) diventa cruciale.

ALGORITMO. Andiamo a creare *progressivamente* l'albero aggiungendo nodi all'albero corrente. In particolare abbiamo il seguente passo iterativo: aggiungiamo l'arco *meno costoso* che connetta un nodo nell'albero corrente ad un nodo *fuori* dall'albero.

Questo passo è ben definito, in quanto connettiamo nodi *interni* con *quelli esterni*, assicurandoci di non creare nessun tipo di ciclo. Infatti, per assurdo se ci fosse un ciclo allora per forza devono entrambi appartenere allo stesso alberi.

Senza dimostrare, enunciamo che questo algoritmo è in grado di *ritornare* anche la soluzione esatta, i.e. la soluzione generata z_g è tale che $z_g = z_*$. ■

Cenni alla Ricerca Locale e Metaeuristiche

X

Migliorare soluzioni greedy: ricerca locale. Esempio di ricerca locale: problema Knapsack, algoritmo k -opt per il problema del commesso viaggiatore (k -opt). Metaeuristiche (cenni).

X

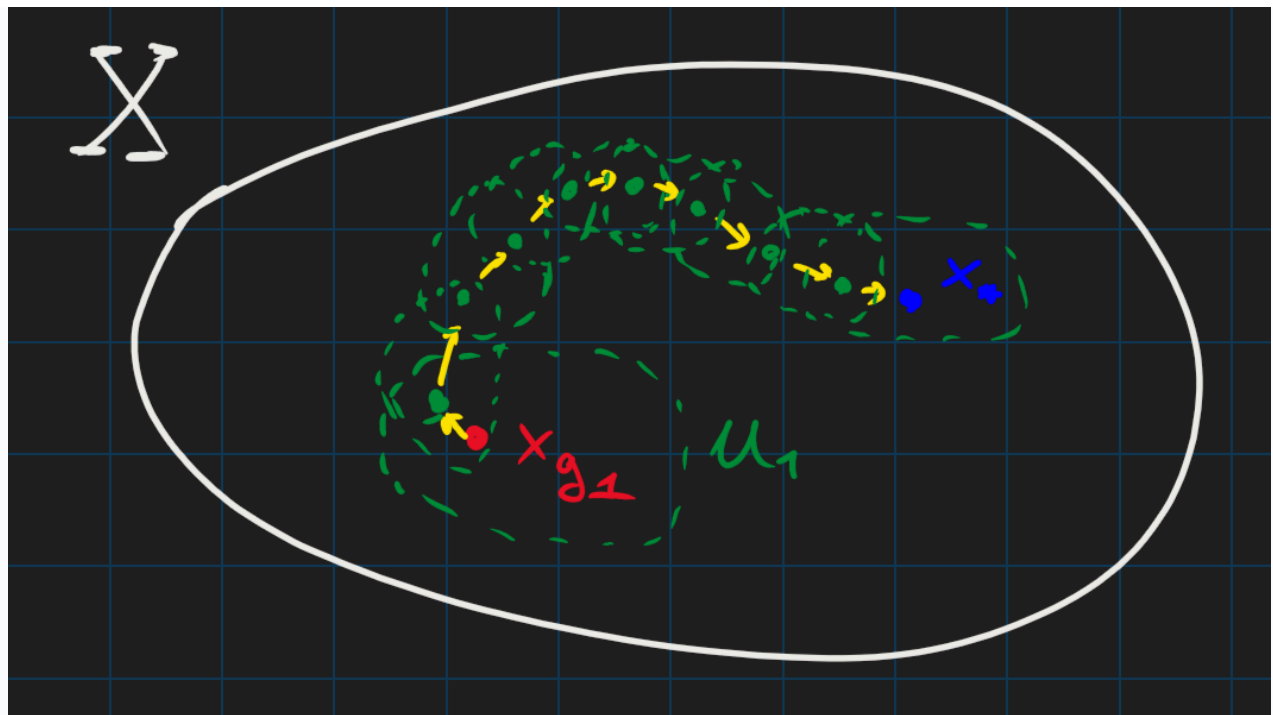
0. Voci correlate

- [Algoritmi Greedy](#)

1. Idea della Ricerca Locale

Q. Data una *soluzione greedy* z_g ad un problema di ottimizzazione, come possiamo *migliorarla* per trovare una soluzione più *"vicina"* alla soluzione ottimale z_* ?

IDEA. Definiamo la *soluzione iniziale ammissibile* dato da un algoritmo greedy z_g come la *"incumbent"* (titolare? boh). L'idea della *ricerca locale* si basa su definire un *intorno* della soluzione x_g e poi trovare la soluzione in quell'intorno. Se la soluzione trovata è *migliore* dell'intorno, allora lo rimpiazza e ri-effettuiamo l'iterazione; altrimenti ci fermiamo e diciamo che la *"incumbent"* è *localmente ottima rispetto al suo vicinato* e l'euristica termina.



X

2. Esempi della Ricerca Locale

Q. Come possiamo definire gli "intorni"?

Vediamo un paio di esempi.

2.1. Problema Knapsack

Dato un problema Knapsack caratterizzato da $n > 0$ oggetti, di peso w_i e di gradimento p_i e ordinati nel senso decrescente rispetto alla densità $\rho_i := p_i/w_i$.

Sia data una sua *soluzione greedy* $z_g \Leftarrow x_g$ con *residuo di capacità* w_r . Allora un modo per *definire l'intorno* di x_g è quella di *effettuare* gli scambi di "oggetti accettati" con "oggetti non accettati" a due a due, tali che si ha una quantità minore di capacità residua.

PYTHON

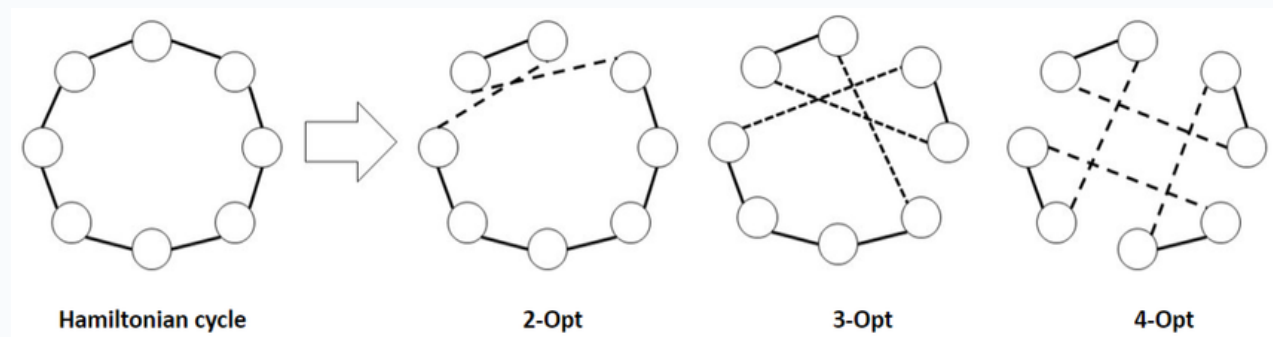
```
def GreedyLocalSearchKP(p, w, xg, zg, cu):
    z = zg
    for i = 1, ..., n:
        if xg[i] == 1:
            for j = i+1, ..., n:
                if xg[j] = 0 and w[i] - w[j] - cu >= 0 and p[j]
> p[i]:
                    swap(xg[i], xg[j])
                    z = z - p[i] + p[j]
                    cu = cu + w[i] - w[j]
```

L'algoritmo è di complessità $O(n^2)$.

2.2. TSP: k -opt

Adesso descriviamo un *algoritmo* per effettuare una *ricerca locale* su una soluzione ammissibile ad un problema *del commesso viaggiatore*, sul grafo $G = (V, E)$.

IDEA. Dato $k > 0$, rimuovere k archi dal cammino hamiltoniano corrente e connettere i risultanti k cammino con archi diversi, per ottenere un cammino diverso. L'intorno del cammino corrente viene definito come tutti i *cammini hamiltoniani risultanti possibili*.



Notiamo che k sarebbe un *"iperparametro"* da determinare a posteriori.

X

3. Metaeuristiche (cenni)

Il problema degli algoritmi di *ricerca locale* è il fatto che una volta che si determina le soluzioni ad essere *localmente ottime*, ci fermiamo. In un certo senso, la mossa di accettare *solamente* le mosse locali ci porta a *"perdere"* un'ottica più lontana, su possibilità di miglioramento.

Gli algoritmi *metaeuristici* vanno a sopperire questo problema, e sono lo strumento più utilizzato per trovare *soluzioni approssimate* di problemi di ottimizzazioni.

Un modo semplice per designare algoritmi metaeuristici è quello di *introdurre* elementi di casualità, uscendo in certi casi pure dalla regione ammissibile.

Esempi:

- Taboo search
- Simulated annealing
- Algoritmi genetici
- Ottimizzazione a colonie delle formiche
- Ottimizzazione "swarm particle"
- Ricerca locale variabile

Algoritmi Approssimati

Algoritmi approssimati. Definizione di algoritmo δ -approssimato. Problema preliminare: problema del matching. Esempi di algoritmi approssimati per il problema del commesso viaggiatore: approssimazione per le MST (alberi minimi coprenti) e l'algoritmo di Cristofides.

0. Voci correlate

- Approssimanti di Soluzioni IP
- Algoritmo Branch & Bound
- Algoritmi Greedy

1. Algoritmi Approssimati

Per risolvere problemi di *programmazione intera* abbiamo visto fin'ora due tecniche principali:

- *Algoritmi esatti*: forniscono la *soluzione giusta* dato una quantità sufficiente di tempo e memoria
- *Algoritmi greedy*: forniscono una *soluzione ammissibile* del problema, senza sapere il suo legame con la soluzione vera

Gli *algoritmi approssimati* sono una "*via di mezzo*" tra gli algoritmi presentati, ovvero sono in grado di ritornare una *soluzione ammissibile* che abbiano un "*marginale d'errore*" con la soluzione ottimale.

#Definizione

Definizione (algoritmo approssimativo).

Dato un problema I su cui si ha il valore ottimale x_*^I , un *algoritmo in tempo polinomiale* $\mathcal{A}$ si dice essere un *algoritmo δ -approssimativo* se vale che la soluzione data $x_{\mathcal{A}}^I$ è tale che

$$R_{\mathcal{A}^I} = \frac{x_{\mathcal{A}}^I}{x_*^I} \leq \delta$$

Per problemi di minimizzazione abbiamo tendenzialmente $\delta \geq 1$, e per problemi di massimizzazione $\delta \leq 1$. Per i problemi di massimo, avvolte considereremo $\frac{1}{\delta}$ come il "*rapporto di approssimazione*".

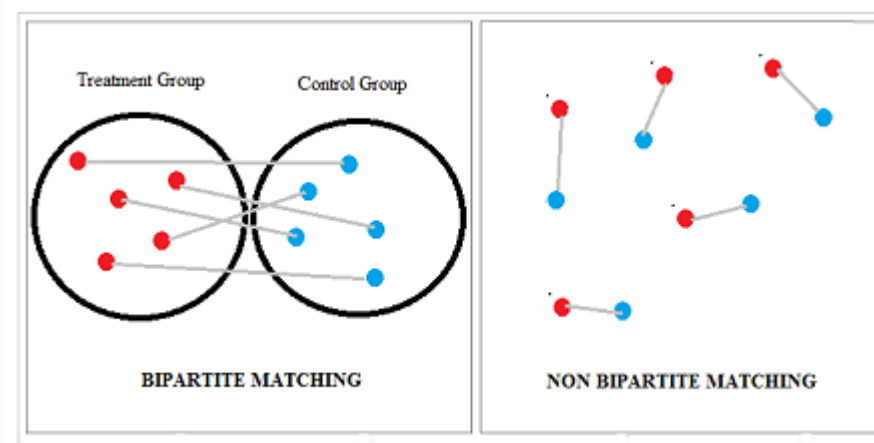
2. Nozioni Preliminari

Prima di vedere esempi di *algoritmi approssimativi*, vediamo un *problema preliminare*.

PROBLEMA. (*Matching*) Nel *problema dell'assegnamento* (*Assignment Problem*) abbiamo che $m + n$ elementi sono partizionati in *due insiemi disgiunti di "persone" e "lavori"*. Se consideriamo invece $2m$ *elementi* in un unico insieme e dobbiamo "*assegnare*" un elemento ad uno e solo un altro, allora abbiamo il *problema del matching*. In particolare la formulazione avviene come segue: sia dato c_{ij} il "*guadagno*" dal matching dei nodi (i, j) , allora

$$\begin{aligned} \max \sum_{i,j} c_{ij} x_{ij} \\ \forall i = 1, \dots, 2n, \sum_{k < i} x_{ki} + \sum_{j > i} x_{ij} = 1 \\ x_1, \dots, x_{2n} \in \{0, 1\} \end{aligned}$$

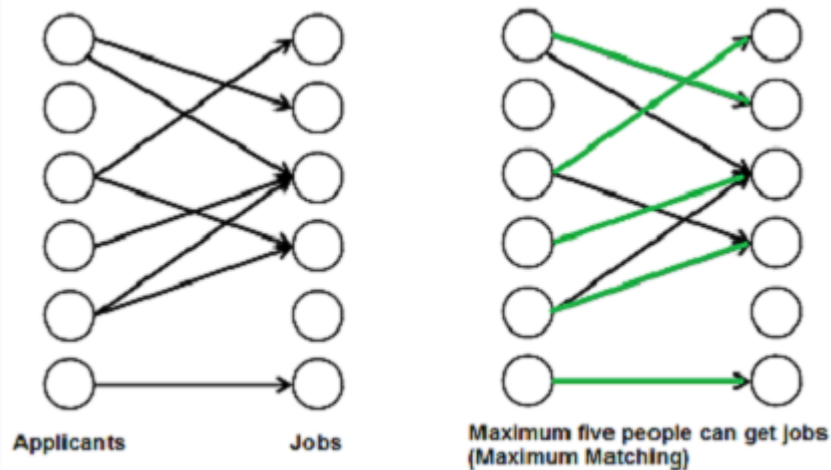
Abbiamo il *problema del matching*.



PROBLEMA. (*Perfect Matching*) Sia dato un *problema di matching* su $2n$ nodi con, questa volta, il "*costo*" c_{ij} . Supponiamo che $G = (V, E)$ dove $V = \{1, \dots, 2n\}$ ed $E \subseteq V \times V \setminus \{i, j \in V \times V : i = j\}$. Se $(i, j) \notin E$, allora sarà *impossibile* creare il *matching* tra i, j e dunque definiremo $c_{ij} := +\infty$. Definendo $\delta(i \in V)$ come i *nodi adiacenti di V*, abbiamo il seguente problema:

$$\begin{aligned} \min \sum_{e \in E} c_e x_e \\ \sum_{e \in \delta(i)} x_e = 1 \\ x_e = 0, 1 \end{aligned}$$

In un certo senso, il *perfect matching* è un ulteriore restringimento sul *problema del matching*, dove imponiamo ulteriori vincoli sulla "*forma geometrica*" del grafo. Tuttavia, in questo caso *non sempre* è garantita l'ammissibilità.



Adesso vediamo l'ultima nozione: *grafi euleriani*

#Definizione

Definizione (grafo euleriano).

Un grafo connesso si dice *euleriano* se tutti i nodi hanno un *grado pari*. In particolare, conterranno sempre un *cammino euleriano*, i.e. contengono un ciclo che passa per ogni arco *esattamente una sola volta*.

X

3. Esempi di Algoritmi Approssimativi

Vedremo due *algoritmi approssimativi* per risolvere il *problema del commesso viaggiatore*, che funzionano sotto l'assunzione della *disuguaglianza triangolare*: ovvero, dato $G = (V, E)$ si deve avere

$$\forall i, j, k \in V, c_{ik} \leq c_{ij} + c_{jk}$$

Ovvero il "*cammino diretto è sempre la più breve*".

3.1. Algoritmo MST

Q. Dato un *cammino Hamiltoniano*, cosa succede se provo a rimuovere un arco?

Succede che ottengo un *albero coprente* sul grafo, infatti si avrebbe un *unico* cammino su tutti nodi. Allora si ottiene subito la maggiorazione

$$\text{MST}(I) < x_*^I$$

Ovvero un *lower bound*. Adesso vogliamo *ottenere un upper bound* utilizzando la MST appena generata, "*ricostruendola*" in un cammino Hamiltoniano.

Si fa con la seguente procedura:

- Visitiamo tutti i nodi *usando solamente gli archi del minimo albero coprente* (ovvero *duplico* gli archi). Lo facciamo mediante una *DFS* sull'albero. In questo modo, la lunghezza è doppia.
- Per la *disuguaglianza triangolare* possiamo "*introdurre*" delle scorciatoie che non fanno aumentare la lunghezza totale dell'attraversamento. Quindi effettuiamo una *DFS*, evitando di creare *archi all'indietro* (possibile per il primo punto).



A questo punto concludiamo, in quanto abbiamo ottenuto un cammino Hamiltoniano. Abbiamo, per il primo punto, sicuramente che la soluzione x_{MST}^I è tale che

$$\frac{x_{\text{MST}}^I}{x_*^I} \leq 2$$

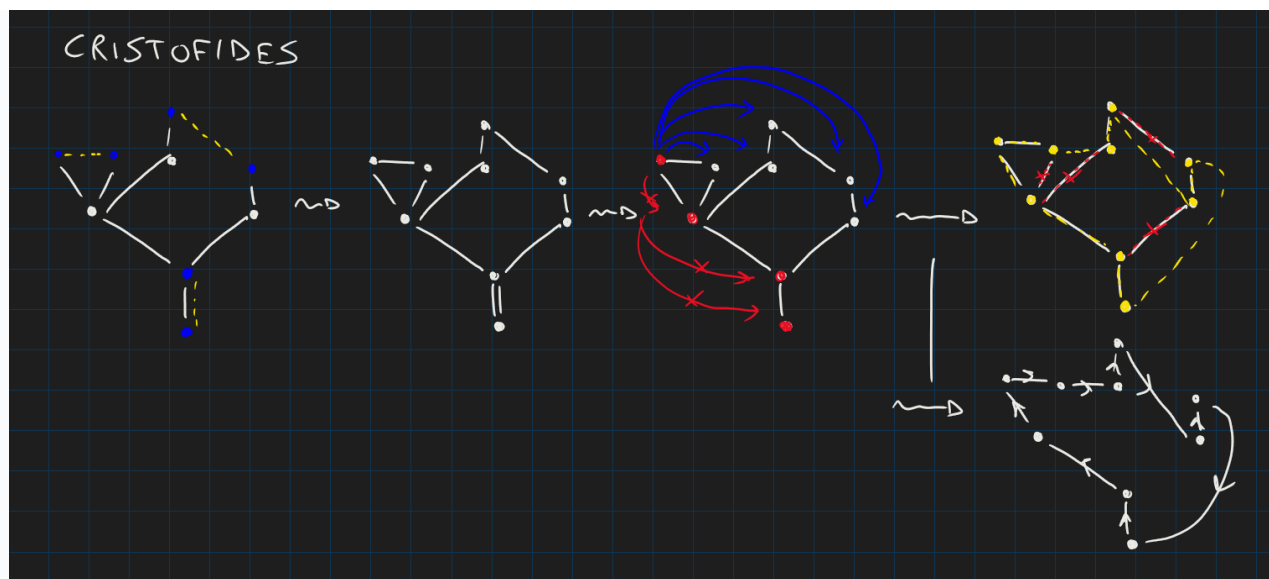
Ovvero la soluzione data dal nostro algoritmo è *al peggio* doppio della soluzione ottimale. $\delta = 2$ è garantito dalla *disuguaglianza triangolare*.

3.2. Algoritmo di Cristofides

Con l'algoritmo di *Cristofides* potremmo fare di meglio, usando le considerazioni preliminari fatte sul problema del matching e grafi euleriani.

IDEA. La procedura è simile all'algoritmo descritto prima, ovvero usando le *MST*, ma facciamo una "*scelta più saggia delle scorciatoie*". In particolare:

- Dato un albero T , certamente alcuni nodi hanno grado dispari: ossia le *foglie*, la *radice* ed altri nodi che hanno "*figli unici*". La somma dei loro gradi dev'essere comunque pari, in quanto la somma di tutti di vertici dev'essere pari (per costruzione)
- Allora *troviamo un matching* tra i nodi di grado dispari, che vada a *minimizzare* la loro somma dei pesi. In questo modo, costruiamo un *grafo euleriano* (in quanto così tutti i nodi hanno gradi pari, siccome abbiamo collegato i nodi dispari).
- Trovando il *cammino di costo minimo* sul *grafo euleriano*, otteniamo un *cammino Hamiltoniano*.



Dimostriamo il seguente risultato:

$$\frac{x_{\text{CRISTOFIDES}}^I}{x_*^I} \leq \frac{3}{2}$$

Supponiamo di avere un *cammino Hamiltoniano ottimale*. Alla prima fase dell'algoritmo, abbiamo un *numero pari di nodi con grado dispari*. Quindi l'algoritmo un *matching* tra nodi di grado dispari; lo denotiamo con M . Notiamo che abbiamo il suo *"matching complementare"*, denotato con M' ; ovvero, $M \cup M'$ andrà a generare un *cammino dei nodi di grado dispari*.

Ma per la disuguaglianza triangolare si ha che (denotiamo $\mu(M)$ la lunghezza del cammino di M)

$$\mu(M) + \mu(M') \leq x_*$$

Pertanto *almeno uno dei due membri della somma* è $\leq \frac{1}{2}x_*$. Scegliamo $\tilde{M}$ come il cammino t.c. $\mu(\tilde{M}) \leq \frac{1}{2}x_*$. Adesso, per ottenere il *costo totale del cammino euleriano* generato dall'algoritmo, dobbiamo anche prendere in considerazione la *spanning tree* T . Tuttavia è certo che $\mu(T) < x_*$, che ci porta al risultato

$$x_{\text{CRISTOFIDES}} \leq \mu(T) + \mu(\tilde{M}) < \frac{3}{2}x_*$$

Concludendo. ■